Journal of Manufacturing Systems 85 (2026) 96–126

Contents lists available at ScienceDirect

Journal of Manufacturing Systems

journal homepage: www.elsevier.com/locate/jmansys

Review

A literature review on deep reinforcement learning for machine scheduling problems

Constantin Waubert de Puiseau [a,*], Furkan Ercan [b], Jannik Peters [a], Marvin Brune [a], Hasan Tercan [a], Christopher Prinz [b], Tobias Meisen [a], Bernd Kuhlenkötter [b]

[a] Institute for Technologies and Management of Digital Transformation, University of Wuppertal, Lise-Meitner-Strasse 27, Wuppertal, 42119, Germany
[b] Chair of Production Systems, Ruhr-University Bochum, Universitätsstrasse 150, Bochum, 44801, Germany

ARTICLE INFO

Keywords:
Machine scheduling
Deep reinforcement learning
Job Shop Scheduling
Machine learning
Literature review
Survey

ABSTRACT

Machine scheduling represents a core challenge in industrial production systems due to its inherently complex combinatorial nature and its critical role in enhancing operational efficiency. With recent advances in artificial intelligence, deep reinforcement learning (DRL) has gained increasing attention as an innovative tool to address scheduling tasks with a self-adapting, data-driven approach. This survey presents a comprehensive review of 143 publications between 2018 and February 2025 that apply DRL to machine scheduling problems. We develop a structured framework to classify and compare problem settings, algorithmic designs, and evaluation methodologies. Key aspects such as action and observation space design, reward functions, neural network architectures, and experimental benchmarks are systematically analyzed. The review identifies current trends, outlines promising patterns, and highlights open research opportunities for DRL-based scheduling solutions. The goal of this survey is to make the rapidly evolving research landscape more accessible to both academics and practitioners and to identify the next steps in research and application. To facilitate reproducible research and customized analysis, we publish the dataset underpinning this review, which includes 61 annotated features per publication, allowing for customizable filtering and further in-depth exploration of niches within the field. This dataset is publicly accessible online.

1. Introduction

1.1. The combination of machine scheduling and deep reinforcement learning

Machine scheduling problems are widespread in industrial production environments and solving them fast and well is a key factor production efficiency. In this paper, we focus on scheduling problems in job shops, flexible job shops, and parallel machine setups, studying static and dynamic, as well as deterministic and stochastic settings. Such problems have in common that they are defined as the challenge of assigning production tasks to machines over time to achieve a certain objective [1]. In practice, they present sequential decision-making problems of combinatorial nature that are notoriously difficult to solve [2].

We can distinguish three general approaches to perform machine scheduling. The first are priority rules, which are commonly used in industry for their fast decision-making capability, ease of implementation, and interpretability [3]. Priority rules, as their name suggests, prioritize between jobs or resources for ad hoc decision-making, either as dispatching rules or within other solution construction logics in combination with simulation models of factory environments. They are fast, but often do not lead to good scheduling solutions and achieved objective values [4–6].

A second solution approach comprises numerical solvers based on mixed integer linear programming or constraint programming [7,8]. Numerical solvers can provide solutions with optimality guarantees, but become intractable for large-scale scheduling problems with respect to the required computational time.

A third general solution approach is the class of metaheuristic approaches [9]. They are often used because they strike a balance between priority rule-based approaches and numerical solvers, achieving intermediate results in terms of both optimality and computation time [10–12].

With the advent of more potent algorithms, cheaper hardware, and increased data availability in the industry, machine learning has emerged as a promising technique for enhancing existing scheduling algorithms by supplementing or replacing stochastic and heuristic decisions within them [13–15]. In particular, deep reinforcement learning

* Corresponding author.
E-mail address: waubert@uni-wuppertal.de (C. Waubert de Puiseau).

https://doi.org/10.1016/j.jmsy.2025.12.017
Received 4 July 2025; Received in revised form 10 December 2025; Accepted 17 December 2025
Available online 12 January 2026
0278-6125/© 2026 The Authors. Published by Elsevier Ltd on behalf of The Society of Manufacturing Engineers. This is an open access article under the CC BY license (http://creativecommons.org/licenses/by/4.0/).

(DRL) has attracted considerable attention for solving sequential decision problems, following its notable successes in domains such as board and video games [16,17]. The declared vision in many publications is to leverage this data-driven machine learning paradigm to address machine scheduling problems, tackling problems of increasing complexity on the basis of more and more granular information to achieve end-to-end DRL [14,18–20]. By capturing patterns in information-rich observation spaces and mapping these to actions that maximize diverse reward formulations, training offline and being applied online, DRL has the potential of achieving real-time adaptability in stochastic, multi-objective environments in which current methods struggle.

1.2. Motivation for this review

Research addressing machine scheduling problems with DRL is rapidly expanding with hundreds of publications, making it progressively more difficult to maintain an overview of the state-of-the-art, trends, and research gaps in this area. As the volume of publications increases, there is a rapid diversification in both the types of scheduling problems studied (cf. Section 5.2.2) and the corresponding solution approaches proposed by researchers. Proposed solution approaches consist of many interdependent design elements, such as the decisions an agent can make, neural network designs, or observation and reward designs. For researchers and practitioners new to the field, it can be challenging to distinguish between different approaches, identify relevant and effective design elements, and assess the extent to which these can be adapted to their specific problem contexts.

The objective of this survey is twofold: firstly, to address the aforementioned challenges and secondly, to make the research field more transparent and accessible. To this end, the survey provides answers to the following research questions:

1. How can different DRL-based solutions be discriminated and which solution designs exists?
2. What types of machine scheduling problems are being researched?
3. What are promising starting points for design patterns when addressing a scheduling problem with DRL?
4. What are the implications of making certain design decisions?
5. What are the major trends and open research gaps in DRL-based machine scheduling?

To address these questions, we compile an up-to-date comprehensive literature corpus of 143 publications and analyze it with respect to the stated questions, as well as meta-information that characterizes the research field and its community. We also publish the raw data from our analysis in table format to give readers the opportunity to filter publications and explore further details. By doing so, we make the following contributions:

- A structured dataset with 61 features per publication and additional qualitative comments
- A comprehensive summary of benchmark datasets and algorithms
- An analysis of promising algorithm designs and current trends
- A framework for the categorization of study design and algorithm design decisions in this research area

1.3. Structure of this review

The remainder of this survey is structured as follows: First, this introduction is concluded with a distinction of this work from related literature reviews and surveys in Section 1.4. Afterwards, the preliminaries of this survey are presented in Section 2, summarizing the used fundamental theoretical concepts and nomenclature of machine scheduling problems (2.1) and DRL (2.2). Subsequently, the procedure for generating the literature corpus is explained in Section 3.

Section 4 provides the details and definitions of the analyzed characteristics that were used to classify the publications in the literature corpus. The entire section is a collection of definitions and is therefore intentionally separated from the results section to keep the latter concise. It starts with explanations of distinctive features regarding the meta-information of articles in Section 4.1. Section 4.2 presents our framework for our classification of scheduling problems. Similarly, Section 4.3 provides the framework and definitions of dimensions along which we characterize the DRL design decisions taken in each publication, and Section 4.4 describes the corresponding dimensions of experimental setups. The results, presented in Section 5, are structured in the same pattern: the results on meta-information are given in Section 5.1, those on scheduling problem characteristics in Section 5.2, those on DRL design decisions in Section 5.3, and lastly Section 5.4 concludes the results with the analysis of experimental setups. Following the detailed presentation of the results, they are discussed in Section 6 with respect to the insights they provide regarding the research questions addressed in this survey. Therein, we also derive recommendations for future work, and discuss the limitations of this survey. Finally, Section 7 offers a comprehensive summary.

1.4. Distinction from related literature reviews and surveys

The large interest of industry and academia in the application of DRL to industrial planning and control problems in general, and to machine scheduling problems in particular, has motivated researchers to conduct numerous literature reviews in recent years. We identified 16 related surveys, each varying in scope and focus. While these works provide valuable overviews of different subdomains, they often leave out essential aspects such as a detailed and discriminative analysis of action and observation space designs, reward shaping strategies, neural network architectures, and experimental setups including benchmark problems and evaluation metrics. Moreover, most existing surveys lack a comprehensive mapping of design choices to specific scheduling problem types, and few provide structured, high-resolution data to facilitate further exploration and reproducibility.

Cunha et al. [21] reviewed the literature on job shop problems solved with DRL available in 2019 and suggested that there was great potential in this combination. However, given the state of research at that time, the review covers only a few papers and is outdated from today's perspective.

Shyalika et al. [22] conducted a review on RL for dynamic task scheduling in 2020. As pioneers of reviewing industrial RL applications, they focused on providing a comprehensive categorization of applied RL algorithms. However, similar to Cunha et al. [21], there were only a few papers available, especially on machine scheduling problems, so the insights for designing such algorithms are, from today's perspective, quite limited.

Wang et al. [23] analyzed a broader literature corpus in 2021, comprising 214 articles on intelligent RL-based optimization methods for manufacturing scheduling. The review features a useful categorization of articles into different action spaces, RL algorithms, and static versus dynamic problem settings. However, compared to the scope of our survey, it includes many methods where RL is only a part of a metaheuristic, which we exclude from our survey. The broader scope in this regard results in a less detailed analysis of design elements compared to our survey.

The research lab around Panzer et al. published two related literature reviews in 2021 [24,25] which, in comparison with the previous reviews, exclusively focuses on *Deep* RL applications. The broader one of the two [24] analyzes 120 systematically collected articles including DRL for production systems, of which 22 articles consider scheduling and eleven consider dispatching, both of which are related to machine scheduling. Along with the categorization into applied DRL algorithms, the authors inform about the addressed application domains and evaluate the level of superiority of the DRL method compared to other

methods. However, a more in-depth analysis of design decisions is omitted due to the broad scope of applications in the review. The second systematic review article by Panzer et al. [25] is more narrowly focused on the applications of DRL in production planning and control. Out of the 38 included articles, 33 deal with machine scheduling problems. Compared to their other review, these articles are described in more detail, especially regarding their applications, objectives, and open challenges. However, as before, a detailed examination of design features is missing.

Waubert de Puiseau et al. [18] present an analysis of reliability considerations in RL-based production scheduling, resulting in a framework for the terminology and metrics of such considerations in a standardized way. As such, they review 28 RL-based approaches in detail with respect to how they consider real-world stochasticity in production systems. However, as in other earlier surveys, many of the reviewed articles do not apply *deep* RL, since the survey was conducted in 2021.

Another literature review first published in 2021 by Kayhan and Yildiz [26] targets RL applications specifically for machine scheduling problems. This comprehensive contribution includes 80 papers and provides a very useful analysis on the objectives, benchmark methods, and action selection schemes used in RL research in the scheduling domain. On the downside, the description of the observation designs is not very detailed, and only about 30% of the methods shown use *deep* learning models.

A survey from 2022 by Esteso et al. [27] on RL analyzing 181 articles about production planning and control considers and summarizes many design decisions and offers a very useful summary for starting practitioners. The majority of the surveyed articles deal with production scheduling papers. However, the authors provide a summary of design decisions rather than an analysis of which design features proved effective in specific problem settings, thus providing limited guidance for context-specific algorithm design.

A year later, Serrano-Ruiz et al. [28] conducted a literature review specifically on DRL for the JSSP focusing on algorithm design and implementation. In addition, they provide a categorization of the reviewed articles into variants of the JSSP, but the analysis covers only 19 articles, showcasing the limited scope of the analyzed literature corpus.

In 2023, Zhang et al. [20] published a survey on the state of the art of DRL-based methods for industrial combinatorial optimization problems, namely scheduling, packaging, and routing. The survey provides a framework for characterizing and comparing the various combinatorial optimization problems, but only includes thirteen studies on scheduling problems.

Modrak et al. [29] reviewed a large body of literature including 1368 articles on the topic of RL for production scheduling. Their statistical analysis clearly highlights the diversity of applications, the rapid evolution in the research field, the main contributors, the trends in scheduling problems that are addressed, and popular scientific outlets. In contrast, we intentionally limit the scope of the problems and methods to enable a more detailed analysis of algorithm design and implementation to derive applicable insights for practitioners.

In a recent survey, Dogru et al. [30] examined the broad field of RL in process industries, specifically analyzing process control, fault detection, and soft sensor derivation through RL. Despite the broad definition of search terms, the overlap with our search is rather small, as they only cover five articles on scheduling.

Smit et al. [15] recently surveyed how graph neural networks are applied to the JSSP and its variants in 58 articles, highlighting the growing trend of modeling machine scheduling problems as graphs to leverage the strengths of these specialized neural networks. Although the survey does not focus on DRL, DRL is identified as the dominant machine learning paradigm for network training. While problem sizes and the performance on benchmark data are analyzed critically, the analysis of other algorithm design choices is not as pronounced. Also, although GNNs are used in many modern DRL approaches for machine scheduling problems, as we will show, other architectures are also the subject of active research.

A recent survey by Zhang and Zhu [31] reviews 93 papers of DRL methods on the JSSP between 2016 and 2023. It provides a helpful categorization into multi-agent and single-agent approaches and rigorously analyzes the trends of applied DRL algorithms. The focus is on key challenges such as growing dimensionality, poor generalizability, and long training times. The authors provide a useful catalog of techniques that may be applied if one of the three challenges arises. Although these challenges will become more relevant as the addressed scheduling problems will move away from the JSSP and towards more realistic problem settings, they are, as we will show, arguably not a major bottleneck in current research on the JSSP. Furthermore, in contrast to the present survey, their review offers limited detail on the specific design choices taken in different approaches.

Shiyong et al. [13] analyze a corpus of literature containing 44 publications that deal with solving the JSSP with DRL concerning design patterns. The authors provide information on many essential design choices about the DRL algorithm as well as the environment, state, action, and reward design. However, the clarity of the discriminatory categories used has certain limitations. For example, the distinction between graph-based and operation-based action spaces is not clearly defined in our view. Furthermore, the lack of temporal analysis of design choices makes it difficult to assess which patterns have proven effective over time. In the present survey, we aim to address these issues by refining the discrimination of several design choices and providing a structured analysis highlighting the strengths and limitations of specific design decisions.

The most recent survey, conducted by Khadivi et al. [14] in 2025, reviews 72 papers on DRL-based machine scheduling. These papers are categorized based on a clear structure by scheduling problem type and their definition of ‘conventional’ vs. ‘advanced’ DRL, mainly separated by the neural network architecture. That is, DRL methods using encoder–decoder or graph neural network architectures are categorized as ‘advanced’ and all others as ‘conventional’. While the survey is very recent and thorough, it does not provide details on design decisions beyond the problem type, optimality criteria, DRL methods, single- vs. multi-agent systems, and the addressed problem size.

The surveys and the extent to which they fulfill the criteria of our survey are summarized in Table 1. The related literature reviews showcase the broad interest of the research community in applying RL to production planning and control tasks, especially machine scheduling tasks, and analyze the related literature according to various criteria and with different levels of granularity. The content of some reviews partially overlaps with our survey. However, none of the reviews alone nor their combination meets all criteria we are aiming to fulfill: an up-to-date large-scale systematic full-text review targeting specifically *Deep* RL algorithms, categorizing and analyzing the individual design choices made for different machine scheduling problems, evaluating their performance with respect to alternative solution approaches, and analyzing the experimental setup used in each study.

2. Preliminaries

This section introduces the nomenclature and categorization used to organize and analyze the literature corpus. It aims to establish a common understanding of scheduling problems and DRL methods by clarifying terms and abbreviations and motivating our analysis feature.

2.1. Machine scheduling problems

Machine scheduling problems are commonly classified using the three-field notation $\alpha|\beta|\gamma$ introduced by Graham et al. [32]. In this notation, α describes the machine environment, β specifies job characteristics and constraints, and γ denotes the optimality criterion. A prominent example is the Job Shop Scheduling Problem (JSSP). In its

Table 1
Comparison of related surveys based on the fulfillment of our desired criteria (○ = no, ◔ = partially, ● = yes). The **year** is the publication year. Surveys are classified as **recent** if they cover papers after 2022. A **large corpus** means that more than 50 publications were analyzed in the survey. We distinguish between merely statistical surveys on meta-information and those analyzing each **full text**. DRL-specific surveys make a distinction between publications on the basis of the machine learning paradigm and the usage of artificial neural networks (NNs) for function approximation. Surveys analyzing **design details** of algorithms beyond the DRL-algorithm and NN architectures are classified as 'partially' fulfilling our criteria. Only if state-, action- and reward-schemes are analyzed, we consider the criteria as completely fulfilled. If any analysis of the performance of algorithms is conducted, the survey partially fulfills the **perf. eval.** criterion, but only reviews that compare the performance between several studies fulfill it completely. Surveys containing information on the experimental setup, i.e., how training and testing is conducted, are flagged as fulfilling the **exp. setup** criterion.

Authors	Year	Recent	Large corpus	Full-text	DRL-specific	Design details	Perf. eval.	Exp. setup
Cunha et al. [21]	2020	○	○	○	○	○	○	○
Shyalika et al. [22]	2020	○	○	●	○	○	◔	○
Wang et al. [23]	2021	○	●	●	○	◔	○	○
Panzer and Bender [24]	2021	○	●	●	○	●	○	○
Panzer et al. [25]	2021	○	●	●	●	◔	◔	○
Waubert d.P. et al. [18]	2022	○	○	●	○	◔	○	○
Esteso et al. [27]	2022	○	●	●	○	◔	○	○
Kayhan et al. [26]	2023	●	●	●	○	●	◔	○
Serrano-Ruiz et al. [28]	2023	●	○	●	●	●	○	○
Zhang et al. [20]	2023	●	○	●	●	○	○	○
Modrak et al. [29]	2024	●	●	○	○	○	○	○
Dogru et al. [30]	2024	●	●	●	○	○	○	○
Smit et al. [15]	2024	●	●	●	○	◔	○	●
Zhang et al. [31]	2024	●	●	●	●	○	◔	○
Wang et al. [13]	2024	●	○	●	●	●	◔	○
Khadivi et al. [14]	2025	●	●	●	●	◔	○	○
ours	2025	●	●	●	●	●	●	●

Table 2
Mathematical notation and its relation to the three-field classification $\alpha|\beta|\gamma$.

Symbol	Description	Relation to $\alpha\|\beta\|\gamma$
$J = \{J_1, \dots, J_n\}$	Set of n jobs	α: Defines the job set
$M = \{M_1, \dots, M_m\}$	Set of m machines	α: Defines the machine environment
$O_i = \{O_{i1}, \dots, O_{in_i}\}$	Ordered set of operations of job J_i	β: Describes job structure and routing
O_{ij}	jth operation of job J_i	β: Operation-level representation
p_{ij}	Processing time of operation O_{ij} on its assigned machine	β: Defines processing characteristics
M_k	Machine on which operation O_{ij} is processed (e.g., in FJSP)	α: Defines machine assignment or flexibility
s_{ij}	Setup or transport time between operations i and j	β: Can include sequence-dependent setup times
r_i	Release time of job J_i	β: May define job release constraints
d_i	Due date of job J_i	γ: Can include tardiness-related objectives
C_i	Completion time of job J_i	γ: Defines performance criteria on job level
$C_{\max}$	Makespan (maximum completion time)	γ: Objective function $C_{\max}$

basic form, denoted as $J|p_{ij}|C_{max}$, the JSSP involves a set of n jobs, $J = \{J_1, J_2, \dots, J_n\}$, that must be processed on a set of m machines, $M = \{M_1, M_2, \dots, M_m\}$. Each job J_i consists of a predefined sequence of n_i operations, $O_i = \{O_{i1}, O_{i2}, \dots, O_{in_i}\}$, where O_{ij} represents the jth operation of job J_i. Each operation O_{ij} requires processing on a specific machine M_k for a given processing time p_{ij}. The objective is to determine a schedule that minimizes the makespan (C_{max}), i.e., the completion time of the last operation across all jobs. The mathematical notation and relation to the three-field notation is summarized in Table 2. The JSSP is notoriously difficult to solve optimally, belonging to the class of NP-hard problems [2]. This complexity stems from the combinatorial nature of the problem, as the number of possible schedules grows factorially ($(J!)^m$) with the number of jobs and machines. Finding an optimal solution requires exploring a vast search space of operation sequences, making it computationally intractable for even moderately sized instances [33].

Often variants of the basic JSSP are considered that go beyond additional constraints and objectives. These include the Flexible JSSP (FJSP), where operations can be processed on alternative machines, introducing a routing component to the problem [34]. Another important extension is the Dynamic JSSP (DJSP), which incorporates stochastic elements such as unexpected job arrivals, variable processing times, or unforeseen machine breakdowns, reflecting the uncertainties of real-world manufacturing environments. Flexibility and dynamic factors integrated lead to the Dynamic FJSP (DFJSP) [35]. The Dual Resource Constrained JSSP (DRC-JSSP) considers not only machines but also other limited resources, such as personnel or tools. Further complexities arise when transportation times between machines and sequence-dependent setup times are included in the model [36].

2.2. Deep reinforcement learning

Reinforcement Learning (RL) provides a mathematical formalism for solving sequential decision-making problems where an agent learns to optimize its solution strategy through interaction with an environment. Problems are typically modeled as a Markov Decision Process (MDP), defined by the tuple $(\mathcal{S}, \mathcal{A}, T, R, \gamma)$, where $\mathcal{S}$ is the state space representing the state of the agent's environment, $\mathcal{A}$ is the action space consisting of all decisions an agent can make, $T : \mathcal{S} \times \mathcal{A} \times \mathcal{S} \to [0, 1]$ is the state transition probability function $P(s'|s, a)$, $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the reward function providing goal-oriented feedback, and $\gamma \in [0, 1]$ is the discount factor weighting future rewards. A state $s_t \in \mathcal{S}$ offers a complete description of the environment at timestep t. The Markov property, fundamental to MDPs, posits that the state transition T depends solely on the current state s_t and action a_t, i.e., $P(s_{t+1}|s_t, a_t, \dots, s_0, a_0) = P(s_{t+1}|s_t, a_t)$ [37].

The agent's objective is to learn a policy, a mapping from states to actions, that maximizes the expected discounted cumulative reward, or

return, over an episode (trajectory) $\tau = (s_0, a_0, s_1, a_1, \dots)$. The return is defined as $R(\tau) = \sum_{t=0}^{\infty} \gamma^t r_t$, where $r_t = R(s_t, a_t)$. Central to many RL algorithms is the state-value function $V^\pi(s) = \mathbb{E}_{\tau\sim\pi}[R(\tau)|s_0 = s]$, representing the expected return from state s under policy π, and the action-value function $Q^\pi(s,a) = \mathbb{E}_{\tau\sim\pi}[R(\tau)|s_0 = s, a_0 = a]$. The action-value function is the expected return after taking action a in state s and subsequently following π. These two functions adhere to the Bellman equations [38]:

$$V^\pi(s) = \mathbb{E}_{a\sim\pi, s'\sim T}[r(s,a) + \gamma V^\pi(s')] \tag{1}$$

$$Q^\pi(s,a) = \mathbb{E}_{s'\sim T}[r(s,a) + \gamma \mathbb{E}_{a'\sim\pi}[Q^\pi(s',a')]] \tag{2}$$

The goal of value-based RL is to find the optimal policy π^* that maximizes the value functions, yielding $V^*(s) = \max_\pi V^\pi(s)$ and $Q^*(s,a) = \max_\pi Q^\pi(s,a)$.

Traditional RL methods store and update state-values or state–action-values in lookup tables. However, tabular representations become intractable in complex domains that are characterized by high-dimensional state and action spaces [39]. Deep Reinforcement Learning (DRL) overcomes this limitation by leveraging Deep Neural Networks (DNNs) as function approximators [40]. Policies (π_θ or μ_θ), value functions (V_θ), or action-value functions (Q_θ) are parameterized by network weights θ, enabling generalization across the state–action space. The specific neural network architectures employed, as discussed in more detail in Section 5.3.5, are critical for effectively processing the complex state representations inherent in scheduling problems.

DRL algorithms can be broadly categorized into **model-free** and **model-based** approaches [41]. Model-free methods learn policies or value functions directly from experience without explicitly learning the environment dynamics T. Conversely, model-based methods learn an explicit model of T and potentially R, allowing for planning (e.g., via Monte Carlo Tree Search as in AlphaZero [42]) which can improve sample efficiency. However, learning models that are accurate enough remains a significant challenge, and inaccuracies in the learned model can lead to suboptimal performance. Due to the complex and often stochastic dynamics of real-world production environments, model-free approaches currently dominate DRL applications in machine scheduling. Within model-free RL, algorithms are further classified based on what they learn, and how different elements are integrated, into the following classes:

Value-Based Methods primarily learn an action-value function $Q_\theta(s,a)$ and derive a policy implicitly (e.g., acting greedily with respect to Q_θ by choosing the action leading to the maximal immediate Q-value). Deep Q-Network (DQN) [40] and its extensions (e.g., Double DQN [43], Dueling DQN [44], Prioritized Experience Replay [45]) are prominent examples that continue to be used and improved. Distributional RL methods, like Implicit Quantile Networks (IQN) [46], extend value-based approaches by learning the distribution of returns rather than just the expectation.

Policy-Based Methods (Policy Gradients) directly learn the parameters θ of a policy π_θ by performing gradient ascent on the expected return objective $J(\theta) = \mathbb{E}_{\tau\sim\pi_\theta}[R(\tau)]$. The REINFORCE algorithm [47] provides the basic formulation, often extended by baselines for action-value estimates for smoothing the reward signal [48]. More advanced methods like Proximal Policy Optimization (PPO) [49] incorporate mechanisms (e.g., clipped objectives or KL-divergence constraints) to promote more stable and sample-efficient policy updates.

Actor-Critic Methods combine value-based and policy-based approaches to combine their respective strengths. An 'actor' network learns the policy π_θ, while a 'critic' network learns a value function (V_ϕ or Q_ϕ) to evaluate the actor's actions, providing lower-variance gradient estimates. Examples of such algorithms include Advantage Actor-Critic (A2C/A3C) [50], Deep Deterministic Policy Gradient (DDPG) [51], and Soft Actor-Critic (SAC) [52], which encourages exploration by incorporating an entropy maximization term into the objective.

Applying DRL effectively necessitates addressing several inherent challenges. Generally, the *sample efficiency* is often low, requiring numerous interactions [39]. Techniques like experience replay improve data reuse [53]. The *credit assignment problem* — attributing delayed rewards (e.g., final makespan) to earlier actions — is pronounced. Careful *reward shaping* can provide denser learning signals, although sparse rewards aligned directly with the ultimate objective (e.g., minimizing the makespan or tardiness) are often preferred to avoid unintended manually induced policy biases [54]. Balancing *exploration* (discovering new strategies) and *exploitation* (leveraging known good strategies) is crucial; methods range from ϵ-greedy in value-based algorithms [37] to entropy regularization in policy-based algorithms or SAC [50]. Ensuring *training stability and convergence* remains critical, addressed through techniques like target networks [55], trust regions [56], and careful hyperparameter tuning. Curriculum learning, where the agent is trained on progressively harder tasks, can also facilitate learning complex scheduling policies [57].

3. Method: Literature corpus generation

To construct a corpus of relevant literature, we carried out a multi-step search and screening procedure inspired by the PRISMA method [58], as illustrated in Fig. 1. On February 25, 2025, we conducted keyword-based searches across four major academic databases: Scopus, IEEE Xplore, ScienceDirect, and Google Scholar. The search query was specifically designed to identify publications focused on the application of DRL to scheduling problems, particularly the JSSP and its variants. The full query is presented in Listing 1.

Listing 1: Search query for literature corpus generation

```
(TITLE(Scheduling)
AND TITLE("Reinforcement␣Learning"))
AND (ABS("Job␣shop")
OR ABS(Machine)
OR ABS(Production))
AND (PUBYEAR > 2017 AND PUBYEAR < 2025)
AND NOT (TITLE-ABS-KEY(*grid))
AND NOT (TITLE-ABS-KEY(cluster*))
AND NOT (TITLE-ABS-KEY(cloud))
AND NOT (TITLE-ABS-KEY(vehicle))
AND NOT (TITLE-ABS-KEY(AGV))
AND NOT (TITLE-ABS-KEY("Maintenance␣Planning"))
AND NOT (TITLE-ABS-KEY("Maintenance␣Scheduling"))
AND NOT (TITLE-ABS-KEY(Bee))
AND NOT (TITLE-ABS-KEY(Genetic*))
AND NOT (TITLE-ABS-KEY("High-Performance␣Comput*"))
AND NOT (TITLE-ABS-KEY(photovoltaic))
AND NOT (TITLE-ABS-KEY(solar))
AND NOT (TITLE-ABS-KEY(PV))
AND NOT (TITLE(suppl*))
AND NOT (TITLE-ABS-KEY(network*))
AND NOT (TITLE-ABS-KEY(swarm*))
AND NOT (TITLE-ABS-KEY(agricultur*))
AND NOT (TITLE-ABS-KEY(FPGA))
AND NOT (TITLE-ABS-KEY(VLSI))
AND NOT (TITLE-ABS-KEY(evolution*))
AND NOT (TITLE-ABS-KEY(robotic*))
AND NOT (TITLE-ABS-KEY(compil*))
AND NOT (TITLE-ABS-KEY(*core))
AND NOT (TITLE-ABS-KEY(edge*))
AND NOT (TITLE-ABS-KEY(iot))
AND (EXCLUDE (LANGUAGE,"Chinese"))
```

The initial search returned 381 records. After removing 66 duplicates, 315 unique publications remained for screening. The screening process consisted of two phases. In Exclusion Phase I, titles and abstracts were reviewed and articles were excluded based on predefined criteria, such as being conceptual frameworks, not focused on production systems, related to robotics, being reviews, or involving location assignment. This resulted in the exclusion of 143 articles, leaving 172 articles for further analysis. Subsequently, in Exclusion Phase II, the full texts of the remaining 172 articles were assessed. Articles were

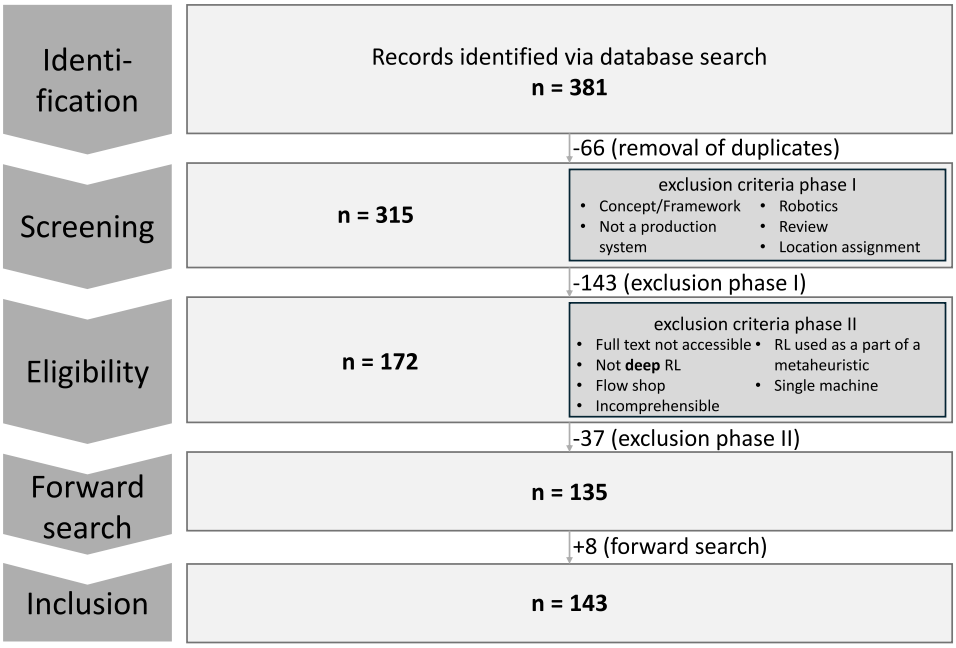


Fig. 1. Flowchart of the literature corpus generation process.

excluded based on several criteria, including inaccessibility of the full text, lack of focus on deep reinforcement learning, emphasis on flow shop problems, incomprehensible content, use of reinforcement learning only as part of a metaheuristic, and focus on single-machine scheduling. This phase led to the exclusion of 37 articles, resulting in 135 publications. While vanilla flow shop problems were excluded, extensions to the flow shop problem, such as flow shops with additional machine constraints, were retained. Finally, a forward citation search was conducted on these 135 publications, which added eight additional relevant articles. Although arXiv was initially excluded due to being a preprint service, relevant arXiv papers identified through this forward search were added. The combination of the screened publications and the forward search resulted in a final corpus of 143 articles for detailed analysis.

4. Method: Analyzed characteristics

We define a set of characteristics according to which all publications are reviewed to address our research questions. These characteristics also form the basis for the discriminative features we report for each publication in the accompanying dataset, and are shown in the mindmap in Fig. 2. The characteristics are analyzed systematically in the present survey, enabling practitioners to identify trends and gaps relevant to their specific problem settings, and offering inspiration for their own algorithm design. We provide a detailed explanation of why and how each criterion is applied in the following. Additionally, we outline our approach for handling cases where the information provided for a given criterion was vague or incomplete.

We begin with the meta-information, i.e. publication years and outlets. Next, we address characteristics of the scheduling problems, followed by characteristics related to the design decisions of the DRL algorithms, and finally, characteristics regarding the experimental setups.

4.1. Meta-information

4.1.1. Year of publication

The publication year is a key criterion for analyzing trends over time. Since trends often correlate with the adoption of newer or more advanced methods that build on previous work, in combination with the other analyzed characteristics, the publication year is valuable for identifying the most promising ideas and methods. We report the publication date of the official online version because it provides a more consistent reference across outlets than the date of the official print publication.



Fig. 2. Mindmap of the analyzed characteristics of publications.

4.1.2. Scientific outlet

A publication's outlet provides valuable insight into its intended contribution — whether to the advancement of machine learning, industry practices, or other domains — and can also serve as an indicator of the article's quality. We categorize outlets into three main types: journal publications, conference proceedings, and arXiv preprints. In addition, we classify these outlets according to their thematic focus.

Specifically, we distinguish between 'Manufacturing', 'Operations Research (OR) and Computer Science', and 'Artificial Intelligence'. The 'Manufacturing' category includes outlets that focus on engineering tasks and technologies relevant to industrial settings. Their titles often include terms like 'manufacturing', 'engineering', or 'production'. 'OR and Computer Science' outlets typically address abstract planning and scheduling tasks, more general solution methods, or applied information technologies in this area. Finally, outlets focusing on 'Artificial Intelligence' aim for advances in basic AI technologies with broader applicability. While this classification may not be entirely objective, it serves as a useful framework for understanding the primary focus of each outlet and thereby roughly the primary focus of each publication.

4.2. Scheduling problem characteristics

Under the umbrella term *scheduling problem characteristics* we unite all characteristics that deal with the problem to which the DRL agent is applied. It comprises the scenario category, the problem type, the corresponding environment behavior, the optimization criterion, the addressed problem sizes, and a classification into online versus offline scheduling schemes. Our distinguished characteristics help to answer, which types of machine scheduling problems are being researched, and help practitioners to pinpoint studies that are relevant to their own scheduling problems.

4.2.1. Scenario category

We broadly distinguish between three categories of scenarios in which scheduling problems are studied and applied: academic problems, real application scenarios, toy scenarios. Academic problems include standardized scheduling problems, such as the classical JSSP or FJSP. These problems often use established or slightly modified benchmark datasets. A real application is characterized by real-world constraints and data from real use-cases, typically drawn from a conducted case study. A toy scenario is one that may be inspired by real-world constraints and conditions beyond academic problems, but is a very clear abstraction from the actual scenario.

4.2.2. Problem type

The problem type is an abstracted problem class like the JSSP, FJSP, DJSP and closely related problem classes, where, for example, artificial due-dates or stochasticity are added. In reference to the three-field notation mentioned before, we discriminate mostly by the machine setup (JSSP, FJSP), i.e., the first field, and ignore further constraints and variations of objectives for this categorization with the exception of dynamic factors (DJSP, DFJSP). We refer to the JSSP and FJSP as static problems and to the others as dynamic counterparts to them. Stochasticity and objectives are analyzed in their dedicated respective subsections of 4.2.3. Note that sometimes, the titles and nomenclature in individual studies can be misaligned with our nomenclature. For example, [59] mentioned 'dynamic job-shop scheduling' but experimented on non-dynamic problems according to our definition.

4.2.3. Environment behavior

In terms of environment behavior, we distinguish between deterministic and stochastic behavior of the environment. If an action taken in a specific state leads to the certain deterministic transition into the next state for each possible state, the environment behaves deterministically. Vice versa, if the same action or sequence of actions can lead to different resulting states, the environment behaves stochastically.

4.2.4. Optimization criterion

The objective of the optimization is an essential component of the scenario category. Although it is often explicitly stated, it is not always equivalent to what the designed algorithm actually optimizes through the reward design. It is therefore important to consider both of these evaluation criteria separately. For example, the stated optimization criterion may be the minimization of the makespan, but the agent is designed to maximize the machine utilization and is punished for taking invalid suggestions, thereby only indirectly optimizing the actual objective metric. We report stated evaluation criteria, not the means to improve them such as the reward. Note that the literature may also contain studies dealing with multi-objective optimization, in which case all objectives are reported, however, without any weightings per objective.

4.2.5. Problem size

The size of a scheduling problem has a significant impact on its complexity and the design of suitable algorithms. It is usually defined by the number of resources (e.g., machines) and operations. However, these absolute numbers are only meaningful when comparing problems of the same type. For example, in a DJSP, the absolute number of jobs arriving over the course of time may not be as important as the frequency at which they arrive, but nevertheless, the number of machines is a limiting factor in any case. This is not comparable to a JSSP, in which all operations are known from the beginning and are also available for processing from the beginning.

Many reviewed articles propose algorithms which can be applied to multiple problem sizes. In that case, we report the largest combination of operations, resources and other problem-specific numbers. Information on the problem size is reported in the online table for reference, but not separately analyzed in this paper due to the difficulty of interpreting problem sizes across different problem types, as explained above.

4.2.6. Online vs. Offline scheduling

Lastly, we distinguish between online or offline scheduling. Typically, in offline scheduling, a complete schedule comprising all information on the problem instance is created. In practice, such a schedule could be used as a fixed plan to follow until the problem diverges too much from the initial one. In contrast, online scheduling is purely reactive, meaning that decisions are made at the point in time when they must be made according to the decision logic. Hence, in practice, online scheduling can be applied continuously and it reacts to changes by design. Although these definitions are quite intuitive, they may cause confusion, when the solution algorithm intentionally creates an offline schedule in a pseudo-online manner. For example, the environment may be represented by a discrete event simulation that temporarily stops when a machine has finished an operation and awaits a machine-selection decision before resuming the simulation. One can think of the agent as taking an online decision. However, the sequence of decisions may simply be recorded to generate an offline schedule (schedule generation) in simulation that is later used for online decisions (schedule application). Especially when dealing with academic problems, in which no distinction between schedule creation and schedule application exists, the online/offline assessment is blurred.

We do not make a distinction based purely on the problem setup, because the same problem setup can be solved either in an online or offline fashion. Instead, the distinction is closely related to the algorithm design, specifically the (inter-)action design between the environment and the agent (cf. Section 5.3.2) and the observation design (cf. Section 4.3.3). In this paper, we classify settings, where an agent performs pseudo-online decision-making while not being given information on the whole problem instance through the observation space as *online* scheduling. We define it this way because, in practice, it could be used for online scheduling, even though it was trained to

create offline schedules. If information of the whole, static problem instance is provided for pseudo-online actions, we classify the scenario as *offline* scheduling. In this way, the problem characteristic is classified from the viewpoint of the agent that makes decisions based on the available observation.

4.3. DRL design decisions

We analyze the following design decisions in the publication of our literature corpus: single vs. multi-agent algorithms, the action space design, the observation space design, the reward design, the neural network architecture, and the DRL algorithm, each with subcategories defined in the respective sections. The analysis of these decisions helps to answer the question, which design patterns are promising starting points when designing a novel algorithm for any machine scheduling problem.

4.3.1. Single- vs. Multi-agent

The decision between a single and multi-agent solution design has many implications for the other design decisions and is therefore a very useful discriminative feature. As the name suggests, in a single-agent setting, only one agent takes scheduling decisions in the global system. Conversely, in a multi-agent setting, multiple agents make decisions. Note that in some articles, multi-agent systems were applied because certain decisions have to be taken in addition to scheduling-related ones, while the scheduling decisions themselves were taken by a single agent [60]. In such a case, we classified the approach as single-agent, since we only focus on scheduling in this review. It is common to make a distinction between cooperative and competitive multi-agent settings [61]. However, only cooperative settings were found in the literature, so this distinction is omitted.

4.3.2. Action space

The action space is a fundamental and multi-faceted design choice that defines the agent's interaction with the environment. In this review, we discriminate between learned vs. enforced action validity, direct vs. indirect actions, single- vs. multi-decision actions, the action logic, and the size of the action space.

Learned vs. Enforced Action Validity There are many action space designs, in which not every action that is theoretically available to the agent is also feasible or valid. For example, if an agent has the autonomy to select a job to process next on a machine, but a job is already finished, the action of scheduling that job again is invalid. Note that in certain problem settings, a sequence of allowed actions leads to a schedule which is only later determined to be invalid. For example, a sequence of actions may result in a schedule in which a job is completed after its delivery date. There are three options to label preceding actions as valid or invalid: first, one may define all actions as valid, since they were executable; second, one may define all actions as invalid, since the result violates a hard constraint; third, as a middle ground, one could try to determine the critical actions leading to the constraint violation and label these as invalid retrospectively. We make use of the first definition, thereby defining all actions executable by the design of the environment as valid. Regarding the implementation, we discriminate between *learned action validity* approaches, in which the agent is punished for invalid actions by a negative reward signal to learn which actions are valid, and *enforced action validity*, in which actions are either valid by the interaction design or otherwise ensured to be valid.

Direct vs. Indirect Actions The direct vs. indirect design choice greatly influences the autonomy of the agent. When actions are *direct*, the agent chooses points at an operation or a resource from a list of all available ones, granting it a lot of control in one decision. Conversely, when actions are *indirect*, the agent chooses from a set of prioritization rules that determine the next operation or resource to use for scheduling. Indirect action spaces may constrain the available decisions to a subset of all theoretically available scheduling choices.

Algorithm 1: Tetris-like logic Pseudocode

```
Result: Fully Scheduled Jobs
while not done do
    op = get_operation_from_action();
    t_m = get_earliest_starttime_on_machine;
    t_j = get_earliest_starttime_of_job;
    earliest_time = max(t_m, t_j);
    schedule(op, earliest_time)
end
```

Algorithm 2: Action-per-timestep Pseudocode

```
Result: Fully Scheduled Jobs
t = 0;
while not done do
    find_startable_jobs();
    if any_jobs_schedulable then
        action = get_agent_action();
        schedule(action);
        if any_jobs_schedulable then
            action = get_agent_action();
            schedule(action);
        end
    end
end
```

Single- vs. Multi-Decision Actions In some scheduling decision settings, multiple decisions have to be made that constitute one executable action. One example is the FJSP, in which both a next job to be processed and the machine on which to process that job have to be determined. In such cases, a single agent can either frame the decision as one composite action representing a job-machine-pair, denoted as *single-decision action* similar to a single-classification task, or an agent takes these actions separately, either in sequence or in parallel as in a multiclass-classification task, which we call *multi-decision action*.

Action Logic Another design choice is the action logic that discriminates ways in which scheduling decisions relate to the environment and to the general solution generation framework. We make a rough distinction between time-based and iteration-based scheduling, each of which encompasses several action logics. The distinction between time-based and iteration-based is best described by a representative example for each.

An intuitive option for the subgroup of time-based scheduling logics takes inspiration from the use of discrete event simulations as environments. We refer to this logic as the *action-per-timestep* logic. Here, the time-discretized process of the simulations is adapted, and the simulation is temporarily put on hold whenever an action by the agent is required to make a decision. Examples for such an event include the events when a machine finishes a process and becomes idle, or when a new job is inserted into the current problem and becomes available for processing. The agent acts from the perspective of making the decision at a particular point in time, and can therefore neither change past events nor schedule operations into the far-ahead future. This action logic is often easiest identified by features in the observation space that hint towards started but only partially finished operations, or features describing that machines are currently in use.

One distinct subvariant is *action-per-timestep, semiautomated*, where, at decision points that theoretically require an action but only one action is feasible, the single feasible action is taken automatically without querying the agent for an action.

Machine scheduling problems may also be indirectly solved through the *order dispatch and fixed sequence* logic, wherein the agent only has

the autonomy to release (dispatch) jobs into the shop floor, and choose from a set of fixed sequences that the job must follow once released. Hence, an action is taken at a concrete point in time but actively and directly defines future paths.

Instead of constructing a single solution step by step through multiple actions, there is also the possibility to determine a single priority rule to create a whole schedule. This option is referred to as *determine pr for whole schedule*, where 'pr' stands for priority rule. Note that in this case, the schedule creation is a single action and an episode consists of solving multiple problem instances.

An intuitive option for the subgroup of iteration-based scheduling is the *Tetris-like* action logic, in which actions can be interpreted as choosing operations which are then integrated into a Gantt-Chart as far to the left as possible without violating any constraints of already scheduled operations. This resembles the way blocks fall down in the game of Tetris. As opposed to the other variants mentioned previously, here, the agent does not act with a notion of taking a decision at a particular point in production time, but rather acts at a particular stage of the schedule assembly process at which a partial solution has been built. Algorithm 1 shows an exemplary pseudocode of how this logic works. Noteworthily, there is no iteration over the current time. In contrast, in the action-per-timestep logic, there can also be no partially finished operations, remaining processing times of an operation, or currently running machines in the observation due to the notion of time. The pseudocode for this general logic is given in Algorithm 2.

Three iterations of exemplary action-per-timestep and Tetris-like action logics are depicted in Fig. 3 to highlight the differences. In the figure, the processes of both interaction logics are depicted starting from an intermediate solution, in which three operations have already been scheduled. In the action-per-timestep logic, the simulation is next stopped at the event when machine 2 becomes idle. The simulation then checks whether there are options to start a new operation or, otherwise, it resumes until the next process is finished. At the next step, the second operation of blue can be scheduled. Therefore, this action is taken and the simulation continues.

There are many other ways in which the action-per-timestep logic can be implemented. Importantly, the action-per-timestep has a notion of past and future. Contrarily, in the Tetris-like action logic, the environment must handle the logic, where an operation can be integrated into a partial schedule. Hence, there is no notion of past and future. Accordingly, the schedule depicted in the last step of the Tetris-like action logic in Fig. 3 would be less likely in the action-per-timestep action logic because the agent would be biased towards scheduling the second blue operation earlier.

A variant of the Tetris-like action logic is the *Tetris-like with left-shifting* logic, where an additional check is computed, whether having scheduled the chosen operation to be integrated prior to the previous operation scheduled on the same machine would have resulted in a smaller makespan of the current partial solution. In that case, the order of the two operations is automatically switched.

The iteration-based action logic option *adding arcs* goes in a similar direction to the 'Tetris-like' logic but is specific to graph representations of scheduling problems. It refers to the logic that previously non-existing or undirected arcs in a graph representation of the (partially solved) scheduling problems are added or their directions fixed. The resulting fully determined graph represents the final solution. A notable difference to the 'Tetris-like' action logic is that, for example, the order of all operations on one machine can be determined before determining the order on other machines. However, the start and end times of operations can still change while keeping the order on a machine the same.

Lastly, another iteration-based alternative approach is to start from an initial solution represented by an ordered sequence of operations on a machine and applying actions that change this sequence. We refer to this approach as *shift of operations in sequence*. Studies in which the action logic could not be determined are marked as 'unknown'.

Size of the Action Space The size of the action space is valuable information for practitioners aiming to understand the implementation of the design. However, it is confusing for the reader to report an exact number, especially in articles where multiple problem sizes are addressed with the same approach. Therefore, a comparison of the action space size between articles is neither feasible nor valuable. Rather, we reported this category in an article-individual and sometimes textual rather than numerical way in the provided dataset, referring to features of the individual action design or problem characteristics. For this purpose, we use the following abbreviations: num_X represents the number of X, e.g., the number of jobs or resources in the scheduling problem. 'pr' stands for priority rule and the number in front of it reports the size of the set of actions from which the agent chooses its action. Where applicable, we provide information about the size of the action space as abstracted mathematical formulas to make them easier to compare to others.

4.3.3. Observation space

Regarding the observation space design, we are particularly interested in two main design choices and the resulting input feature vector size.

Raw vs. Engineered Features The first fundamental design choice is related to the feature generation, namely whether or not the DRL agent receives a problem description of only *raw features*, such as the processing time and required machine per operation, which together constitute the precise definition of a problem instance without redundant information. The alternative is *engineered features* that requires additional computations, such as the sum of all remaining processing times of a job. In this definition, we do not treat common normalization techniques as 'additional computations'. Note that the two alternatives are mutually exclusive, as we treat an algorithm using raw features and engineered features as relying on engineered features.

Fixed vs. Flexible Observation Size The second design choice to be made is whether the observation space has a *fixed size* or *flexible size*, enabling the agent to, for example, only perceive information on the remaining tasks or available machines during the solution generation, or enabling the same agent to solve problems with differing numbers of jobs and machines. Note that an observation space design of flexible sizes requires a neural network architecture that can process different input dimensions, such as recurrent, graph or self-attention-based neural networks.

Myopic Observations In addition to the two fundamental design choices, we classify whether the observation space design is myopic or not. A myopic observation is defined as only providing limited information on the next possible operations or resources that do not take into account the rest of the current problem. For example, providing a machine-centric agent only with processing times of operations in its buffer would be myopic. Providing information on the required machine per operation would also count as myopic. In our strict definition, providing the sum of remaining processing times in a job for each operation in its buffer would include future information and therefore be classified as non-myopic. Note that minimal information entered per node in a graph neural network still provides non-myopic information through the graph structure and multi-step message passing, as in [62]. In summary, observations are myopic if they:

- only contain information about the immediate next actions (e.g., operations or machines),
- contain non-comprehensive information,
- are non-comprehensive even if individual inputs are combined.

Size of Observation Spaces As for the size of the action space, we provide numbers of formulas defining the size of the action vector. The designs and definitions vary too widely to allow a useful statistical analysis. However, the numbers and formulas are provided in the literature table for reference. Therein, the number of job or resource

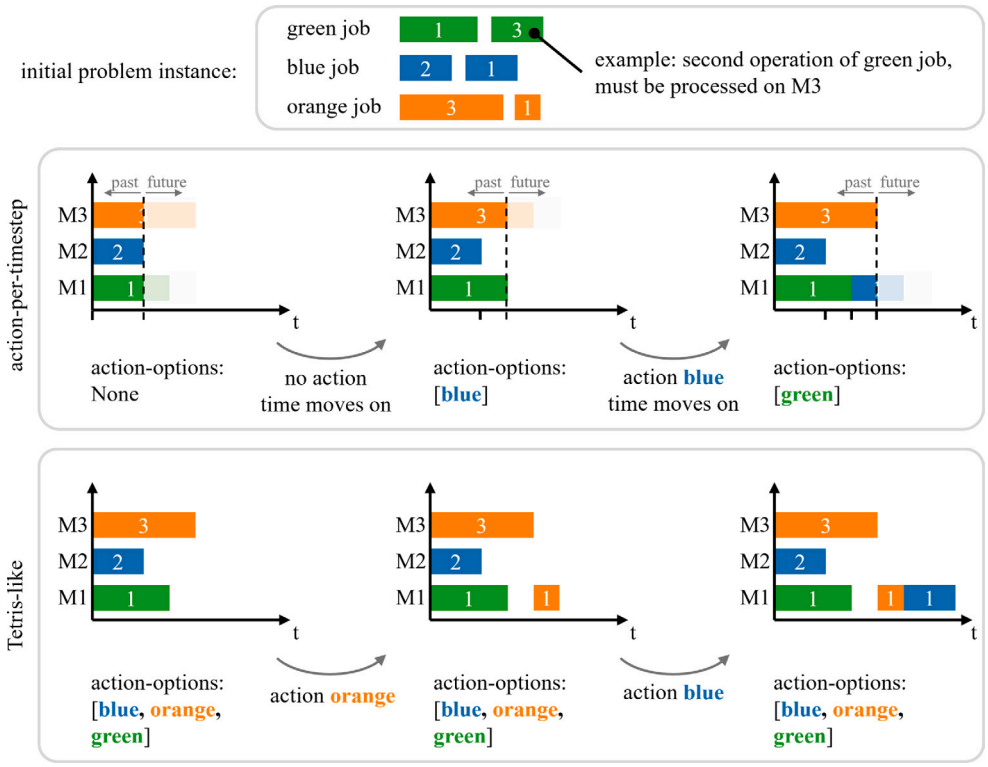


Fig. 3. Three exemplary iterations of the action-per-timestep and Tetris-like action logics.

'X' is reported as 'num_X'. In some cases, the provided description or references to other descriptions in publications are insufficient to define the exact number of features [63], even if many details are provided on the integration of such features into an architecture [11,64]. When numbers or other information are missing, the corresponding table entry is marked with '?'. When only certain information is missing within a formula, such as the exact job features, but one can infer that the observation space includes this information 'x' for every job, we report them as $x \times num_obs$. Whenever possible, missing information was extracted from provided code, as for [65]. Where fixed numbers of features are reported, the observation includes averaged features or similar techniques [66], such that the observation space would not change in size for a different problem size.

4.3.4. Reward design

The reward design is another crucial aspect in the DRL algorithm design. The questions, whether invalid actions are penalized or not, and which criterion is optimized, are already answered in other sections. Of course, they need to be taken into account to understand the whole reward design. Here, we only further discriminate between *dense rewards* and *sparse rewards*. Dense rewards are given when each state transition is rewarded, punished, or at least evaluated, such that non-zero rewards are theoretically possible for each step. Note that despite the employment of dense rewards, many zero-valued rewards may still occur. Sparse rewards, on the other hand, intentionally assign zero-value rewards for what are then considered intermediate steps. By our definition, sparse rewards span from intentionally only evaluating every second step to only evaluating the final step.

4.3.5. Neural network architecture

The choice of neural network architecture also has a great impact on the overall DRL design and performance. We discriminate between fully connected networks (FCNs), convolutional networks (CNNs), recurrent neural networks (RNNs), graph neural networks (GNNs), and self-attention based architectures (ATTN) that build the basis of transformer architectures. Elaborate architectures may combine multiple of these elements in one neural network, multiple networks with different architectures may be used for one agent, or multiple networks may be used for comparison within one study. We conduct our classification of the network architectures as follows, leaving the opportunity for one publication containing multiple components open: Almost every neural network contains a fully connected component. To avoid redundancy, we therefore only classify those networks as *FCNs* that contain none of the other components. At the same time, whenever no explicit architecture was described by the authors we assumed that default FCN networks were used. Networks containing at least one convolutional layer are classified as *CNN*. If a network contains at least one recurrent unit, for example a gated recurrent unit or a long-short-term-memory unit, it is classified as an *RNN*. All networks employing a self-attention mechanism are classified as *ATTN*. *GNNs* introduce an exception, since they may contain convolutional elements as well as attention-based elements. A common denominator, however, is that they all rely on message passing between nodes of a graph representation of the scheduling problem. Hence, we classify all network architectures borrowing from the graph representation for message passing as GNNs, but neither classify a convolutional GNN as CNN, nor a graph attention network as ATTN.

4.3.6. DRL algorithm

Regarding the DRL algorithm, we make a rough distinction between *value-based* and *policy-gradient-based* algorithms, classifying hybrid algorithms like soft-actor-critic (SAC) or deep deterministic policy gradient (DDPG) and its subvariants as policy-gradient-based. When it is clearly identifiable, we also report the name of the specific DRL algorithm for easier reference.

4.4. Experimental setup

In addition to the problem description and algorithm design, details of the experimental setup are essential for assessing the relevance,

Table 3
Benchmark instances for scheduling problems.

Abbr.	Alias	Problem	Largest Inst.	#Inst.	Ref.
TA	Taillard, TA	JSSP	100 × 20	80	[68]
DMU	Demirkol	JSSP	50 × 20	80	[69]
ABZ	Adams, Balas and Zawack	JSSP	20 × 15	5	[70]
FT	Fisher and Thompson	JSSP	20 × 5	3	[71]
LA	Lawrence	JSSP	30 × 10	40	[72]
ORB	Applegate and Cook	JSSP	10 × 10	10	[73]
SWV	Storer, Wu and Vaccari	JSSP	50 × 10	20	[74]
YN	Yamada and Nakano	JSSP	20 × 20	4	[75]
DCT[a]	Da Col and Teppan	JSSP	1000 × 1000	90	[76]
MK	Brandimarte	FJSP	30 × 15	15	[77]
HU	Hurink, LA, FT	FJSP	30 × 10	120	[67]
BE[a]	Behnke	FJSP	100 × 60	45	[78]
JFS	SJFS, MJFS	FJSP	12 × 8 × 4	20	[79]
DPpaulli	Dauzere	FJSP	20 × 10	18	[80]
BA[a]	Barnes	FJSP	15 × 18	21	[81]
HAM	Ham	FJSP + AGV	30 × 10	120	[82]
TA(flow)[a]	TA FSS	Flow Shop	500 × 20	115	[68]
DW1[a]	–	PMFS[b]	$n = 30,\ m = 5,\ f = 8$	–	[83]
DW2[a]	–	PMFS[b]	$n = 80,\ m = 5,\ f = 16$	–	[84]
LIAO[a]	–	PMFS[b]	$n = 80,\ m = 9,\ f = 16$	–	[85]

[a] Own abbreviation.
[b] PMFS = Parallel Machines with Family Setups.

comparability, and reproducibility of an algorithm's results. Hence, this information is essential for answering the question, which design patterns are promising candidates for future developments. Therefore, we report whether benchmark datasets and baseline algorithms were used, whether the generalization capabilities of the proposed methods were evaluated, and whether the source code is publicly available.

4.4.1. Benchmark datasets

Evaluating new algorithms on established benchmark datasets is a widely accepted practice to validate performance and ensure comparability. We classify each publication based on whether it includes an evaluation on such a benchmark. Table 3 provides an overview of all benchmark datasets referenced in the literature reviewed. For each dataset, we list commonly used aliases for ease of recognition, the associated scheduling problem type, the size of the largest instance, and the total number of problem instances. In the case of 'Identical Parallel Machines with Family Setups' (PMFS), instance sizes are defined by the number of jobs n, machines m, and family setups f. We found that for some datasets, only a description of the dataset generation process exists rather than exact publicly available data. We still include such instances in our benchmark-based evaluations, given their standardized use within the respective studies.

Note that when studies diverge from typical vanilla benchmark scenarios such as the JSSP and FJSP, authors sometimes augment the standard benchmark datasets with additional information. For example, one may use the TA dataset but define custom release times for each job to consider a dynamic JSSP. While the final results are not comparable to the original benchmark data, we still label such cases as using the TA benchmark, because it is more informative than generating new test data from scratch, and such data may be more easily adopted by others. One noteworthy FJSP dataset is HU [67], which adapts the jobs, operations and processing times of the TA and FT JSSP benchmarks, but introduces more machine flexibility in three degrees, the *edata*, *edata* and *vdata* versions. In *edata*, only a limited number of operations may be processed on more than one machine; in *edata*, most operations may be processed on multiple machines; and in *vdata*, every operation may be processed on multiple machines.

4.4.2. Benchmark algorithms

In order to demonstrate the relative performance of proposed algorithms, researchers commonly compare them against established baseline algorithms. These typically include heuristic priority rules, metaheuristics, exact solvers, third-party DRL-based methods, and variations developed by the authors themselves.

Priority Rules A summary of the 16 most frequently cited priority rules identified in the surveyed literature, especially those cited more than four times, is provided in Table 4. Less frequently cited priority rules are listed in Table A.9 in the Appendix A. Each table lists synonyms found in the literature for each rule, along with a brief description. It is important to note that we only report job sequencing and machine choice rules and explicitly exclude transportation rules.

Unfortunately, rule names do not always unambiguously identify the underlying method. For example, a study might refer to the use of the 'Most Work Remaining' rule, which means jobs are prioritized according to the sum of remaining processing times until completion. However, depending on the action logic, this may or may not include the remaining processing time of a job currently being processed. Where we can, we will therefore provide further details on each rule in the provided online table. Moreover, the 'Most Work Remaining' rule can be interpreted from a job's perspective, as defined above, or from a machine's perspective. Hence, for maximal consistency, we examined exact definitions and formulas whenever available, and assigned each publication to the rule and name that most closely matched the majority usage.

Many authors construct composite dispatching rules by combining multiple individual rules. In the simplest case, a primary rule is used for prioritization, with a secondary rule applied as a tiebreaker. In other cases, rules are combined mathematically by (weighted) summation, multiplication, or division. A third type of composite dispatching rules is used in problem settings with multiple decisions per step, such as the FJSP, where, for example, the job and machine selection may independently follow a rule like 'Shortest Processing Time'. Due to the absence of a clear and concise way to indicate how rules are combined in each study, we only indicate if an elementary dispatching rule is included in a study, regardless of whether it is used independently or as part of a composite method. For the reasons outlined above, readers are encouraged to consult the original publications for a detailed understanding of the specific benchmarks and rule implementations used. Detailed information on many rules can be found in the works by Haupt et al. [3] and Sels et al. [86].

Metaheuristics serve as relevant benchmark algorithms because they represent established optimization methods known to yield high-quality solutions for complex scheduling problems within reasonable computational timeframes, providing a strong baseline against which novel DRL approaches can be evaluated. They generally function as sophisticated black-box search algorithms, often using stochastic elements to explore vast solution spaces effectively without requiring detailed

Table 4
Common priority rules in production scheduling mentioned more than five times in the literature corpus.

Rule	Explanation	Synonyms
SSU	Shortest setup time first	–
MWKR	Most work remaining	MTWR, LRM, LRD, MPTR, LTPT, LWT, JLRPT
MOR	Most operations remaining	MOPNR, JMOR, MPSR, JMRJ
FDD	Flow due date: release date + total processing time	–
FIFO	First in, first out	–
LWKR	Least work remaining	SJN, STPT, SRM, SRD, LWR, SRPT, LPTR, JSRPT, LWT, SWT, SROT, SJF
LOR	Least operations remaining	FOPNR, LPSR, LRO, LOPNR, JLRJ
LQNO	Least queue next operation: fewest jobs in queue of the next machine	WINQ, LQS
MQNO	Most queue next operation: most jobs in queue of the next machine	NINQ, LQE, SQ
Random	Randomly selected from equal distribution	–
LIFO	Last in, first out	–
LPT	Longest processing time	LSO, JLPT
MST	Minimum slack time (due date - remaining processing time)	MS, STOP, LSR, MDD, SLACK
CR	Smallest critical rate (time until due date divided by remaining processing time)	–
SRM	Shortest remaining machining time (for jobs being processed)	JSWT
LRM	Longest remaining machining time (for jobs being processed)	JLWT, EET

problem-specific knowledge. Metaheuristics are broadly classified into single-solution-based methods (iteratively improving one candidate) and population-based methods (maintaining and evolving a set of potential solutions) [9].

Common population-based algorithms are Genetic Algorithms (GAs). The search process begins with defining an effective chromosome (genotype) representation [87]. This encoding dictates how the search space is explored. Representations are typically direct (e.g., operation-based, where a permutation of operations defines the processing sequence) or indirect (e.g., priority rule-based, encoding a sequence of rules applied dynamically during schedule construction) [88]. The subsequent search procedure depends on the genetic operators — selection, crossover, and mutation, as well as their parameter tuning [9]. Selection chooses individuals for reproduction based on fitness (e.g., using ranking, tournament, or roulette wheel schemes, often with elitism). Crossover combines genetic material from selected parents to generate offspring, using techniques like one-point, two-point, order crossover (OX), or precedence preserving order-based crossover (POX) for scheduling problems. Finally, mutation introduces random alterations to offspring chromosomes, maintaining population diversity and aiding search space exploration, with operators like swap, insertion, or problem-specific mutations. The choice and parameterization of these operators critically impact the algorithm's search effectiveness and convergence [89,90]. It is important to note the chromosome itself (e.g., an ordered list of operations in an operation-based representation) is not the final schedule but guides its construction. The final schedule is generated in the decoding step, respecting inherent problem constraints (such as JSSP's operation order and machine non-overlap), converting the abstract genetic representation into a concrete, assessable solution. This step also enables the calculation of the schedule's fitness (see Section 5.2.4).

The simple 'GA' label encompasses considerable variety. To illustrate this, Table 5 lists numerous GA variants cited by authors, ranging from established multi-objective approaches like NSGA-II to specialized hybrid designs. This diversity highlights that direct comparisons based solely on the use of a 'GA' benchmark should be approached carefully.

Table 5
Overview of genetic algorithm variants and references.

Variant	Introduced in
Vanilla GA	[91]
Effective GA	[92]
NSGA-II	[93]
2SGA	[94]
GA with AC and TS	[95]
GA with Decision Tree	[96]
GA with SVM	[97]
Self learning GA	[98]
Hybrid heuristic GA	[99]
GA with PDR	[100]
GA-based Hyper Heuristic (GAHH)	[101]
Multi Objective GA based on lattice design	[102]
NMHPGA	[103]
Improved niche GA (INGA)	[104]
Improved GA	[105]

While GAs are the most common, other metaheuristics like Tabu Search (TS), Ant Colony Optimization (ACO), and Particle Swarm Optimization (PSO) are also used. These other metaheuristics operate on different core principles: TS leverages a memory-based neighborhood search to escape local optima, ACO simulates pheromone trails deposited by multiple agents to guide probabilistic solution construction, and PSO models a swarm of candidate solutions whose movements through the search space are influenced by individual and collective best-found positions.

Solvers In addition to priority rules and metaheuristics, exact numerical *solvers* are occasionally employed. We report whether a study reports solver-based solutions and specify the solver used when available. In this case, it is important to note that some authors compare their solutions to known optimal ones from benchmark instances without applying a solver themselves.

Third-party related DRL-based methods Another category of benchmark algorithms includes state-of-the-art DRL methods. We consider a study to benchmark against such a method in three cases: if the results on benchmark data are reprinted, if the algorithms are reimplemented and directly compared on the same hardware, or if substantial parts of the solution are reused from a third-party algorithm with only minor adaptations (e.g., applying a learned construction heuristic in a dynamic rather than a static setting).

Own DRL-Agent In addition to externally developed benchmark algorithms, researchers sometimes report results of alternative versions of their own solution designs, such as different DRL variants, to highlight the importance of specific design choices, following an ablation study approach. Unlike external benchmarks, these comparisons provide less insight into the absolute performance of the method and more insight into its sensitivity to internal design decisions. We acknowledge such ablations as benchmark algorithms, but categorize them separately as *own DRL-Agents*.

4.4.3. Generalization ability

Generalization of machine learning models typically refers to the performance of a trained model on data it has not seen during training. For models that perform scheduling, we have observed four different grades of generalization ability concerning how large the difference between the training and testing scenarios is. The first grade is *no generalization*, meaning that a model is trained on the same problem instance it is subsequently used on, in a genetic algorithm manner. The second

Table 6
Mapping between analyzed information definitions, result sections, and corresponding figures and tables.

Analyzed information definitions	Results description	Figures and tables
Article Meta Information Section 4.1	Section 5.1	
Year of Publication Section 4.1.1	Section 5.1	Fig. 4
Scientific Outlet Section 4.1.2	Section 5.1	Fig. 5
Scheduling Problem Characteristics Section 4.2	Section 5.2	
Scenario Category Section 4.2.1	Section 5.2.1	
Problem Type Section 4.2.2	Section 5.2.2	Fig. 6
Environment Behavior Section 4.2.3	Section 5.2.3	Table 7
Optimization Criterion Section 4.2.4	Section 5.2.4	
Problem Size Section 4.2.5	Section 5.2.2	
Online vs. Offline Scheduling Section 4.2.6	Section 5.2.5	Fig. 7
DRL Design Decisions Section 4.3	Section 5.3	
Single- vs. Multi-Agent Section 4.3.1	Section 5.3.1	Fig. 8
Action Space Section 4.3.2	Section 5.3.2	Fig. 9, Fig. 10, Fig. 11
Observation Space Section 4.3.3	Section 5.3.3	Fig. 12, Fig. 13
Reward Design Section 4.3.4	Section 5.3.4	Fig. 14
Neural Network Architecture Section 4.3.5	Section 5.3.5	Fig. 15, Fig. 16
DRL Algorithm Section 4.3.6	Section 5.3.6	Fig. 17, Fig. 18
Experimental Setup Section 4.4	Section 5.4	
Benchmark Datasets Section 4.4.1	Section 5.4.1	Fig. 19, Fig. 20, Fig. 21
Benchmark Algorithms Section 4.4.2	Section 5.4.2	Fig. 22, Fig. 23, Fig. 24, Table 8
Generalization Ability Section 4.4.3	Section 5.4.3	
Code Availability Section 4.4.4	Section 5.4.4	

grade performs a *train-test split*, meaning that training and test data are distinct. This can be achieved by sampling problem characteristics from random distributions. Notably, dynamic scheduling problems, in which new jobs continuously arrive according to a random distribution, making each training episode unique, and in which an agent is continuously trained and deployed, are considered to inherently include a train-test split. We treat grades one and two as mutually exclusive categories, although active search, a technique in which a generalized trained model is subject to targeted training at test time [106,107] could be viewed as a hybrid between the two. Cases involving active search are categorized as grade two generalization ability in our classification. Grades three and four are considered extensions of grade one. Thus, any algorithm classified as grade three or four is automatically also assigned to grade one. Grade three is the *intersize transfer*, meaning that a model was trained on instances with a certain number of machines and/or jobs but is also tested on problem instances with different numbers of machines and/or jobs. Grade four generalization ability is *zero-shot testing*, meaning that test data contains different properties than the training data in other aspects than grades one or two, such as different processing time distributions, introduced stochasticity in the environment, or new constraint constellations.

4.4.4. Code availability

All publications that include a link to a publicly available code repository are considered to provide sufficient code and data to reproduce their results. We did not verify whether all provided code executes successfully, but confirmed the presence of an environment and agent implementation in each repository. Where available, we recorded the corresponding web links and verified their accessibility as of February 24, 2025.

Table 6 provides an overview over the definitions that were given in this section and the corresponding sections and specific figures and tables discussing the results.

5. Results

In this section, we present the results of our analysis of the above criteria in the same order.

5.1. Analysis of article meta-information

The trend in the number of published articles per year over time is illustrated in Fig. 4. As shown, the annual publication count has steadily increased, from just two to five publications in 2018 and 2019, to 36 in 2023. It is reasonable to expect an even greater number of publications in 2024. The relatively lower count for 2024 may be attributed to articles that had not been published and indexed in the corresponding databases at the time of the literature search conducted in early 2025.

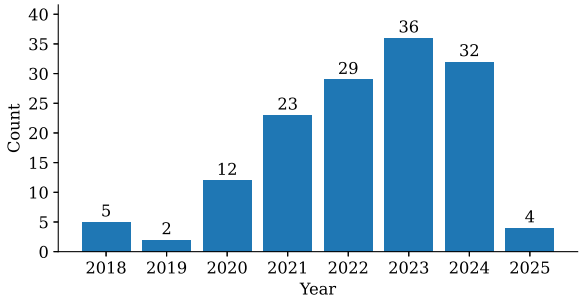


Fig. 4. Number of published articles per year on DRL-based machine scheduling over time.

Next, we analyze the distribution of scientific outlet types to identify the communities most engaged in advancing the field and the types of contributions being made. Fig. 5 presents a list of journals and conferences that have published at least two articles. The remaining outlets are grouped under 'other'. The bars next to each outlet's name indicate the number of articles published, with the color of the bars representing the categorized topic focus of the outlets. A significant 65% of articles are published in scientific journals. Overall, 81 distinct outlets published the 143 articles, highlighting the large variety of research fields contributing to the interdisciplinary field of DRL-based scheduling. Notably, only a few journals have published more than five articles, suggesting that no single outlet has yet emerged as the dominant platform for contributions in this research area.

5.2. Analysis of addressed scheduling problems

5.2.1. Analysis of scenario categories

We divided the literature into those publications addressing real-world applications, toy scenarios or academic scheduling problems. Academic problem settings make up 65/143 (45.4%) of all studies and were either used early on as proof-of-concepts that scheduling with DRL can work, or to design novel algorithm elements to increase scheduling performance in a comparable way. We found that even more studies,

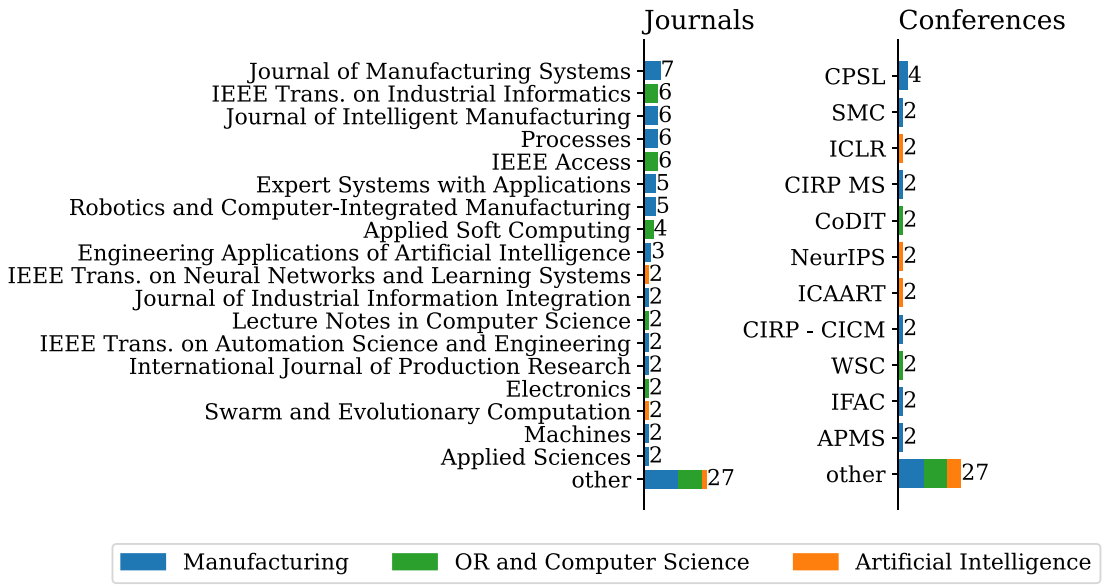


Fig. 5. Number of publications in different scientific outlets.

72/142 (50.7%) of all studies, work on toy scenarios. The large number of toy scenarios indicates the progression of the research field towards handling increasingly complex scenarios. Real-world applications are still few (10/143, 7.0%), highlighting that DRL is a novel solution alternative that has not yet made a breakthrough to be applicable to the often very complex real-world settings. It should also be noted that most studies in this category still do not take all elements in the real-world setting into account. However, the studies still work as valid proof-of-concepts, document valuable lighthouse projects, and give an impression of how advanced this research field currently is.

5.2.2. Analysis of problem types

We analyzed the number of publications per scheduling problem category, and how the numbers change over time. The corresponding numbers are plotted as an explored bar chart in Fig. 6. Averaged across the years, the static JSSP and FJSP are equally represented, making up 28.8% and 29.0% of all studies, respectively. Their dynamic counterparts, DJSP and DFJSP, are studied less frequently in total but with a similar distribution, making up 13.7% and 15.1%, respectively. Upon closer inspection, we observed certain trends over time. Whereas the JSSP and DJSP (blue color) were the dominant objects of study until 2021, the trend changed in 2022 and 2023, before their flexible versions (orange color) could be found more often. Concurrently, the ratio between the JSSP and DJSP shifted slightly but gradually from a higher proportion of static problems to dynamic problems. This trend was not observed in the flexible problems. Nevertheless, the general trend of moving from JSSP to flexible versions, coupled with the shift towards dynamic problems, equates to a change in the nature of the challenges faced by DRL agents. This interpretation is promising, as it marks progression in the research field towards dealing with real-world applications.

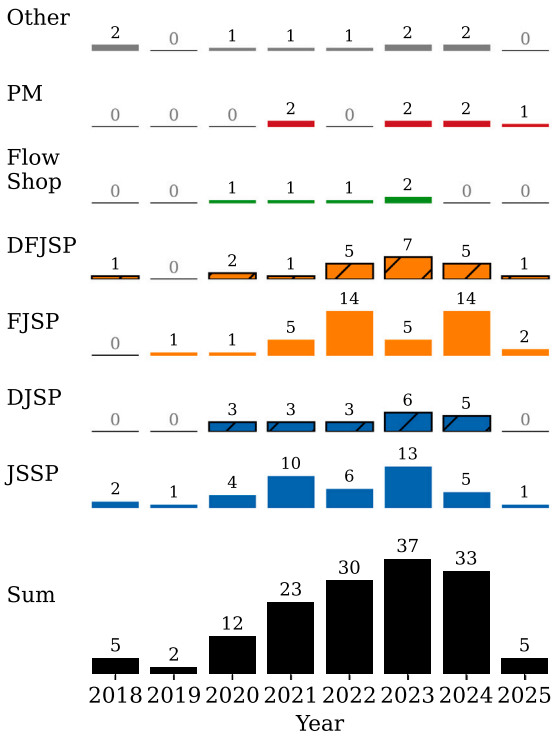


Fig. 6. Scheduling problem categories addressed in the studies over time. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

5.2.3. Analysis of environment behavior

The environment behavior, classified as static or dynamic, by nature correlates strongly with the static and dynamic problem settings in which new jobs arrive. However, they are not identical, because the DJSP includes job releases that can be pre-defined rather than stochastic. Stochastic job arrivals, job releases into the system, or job cancellations were found 37 times. The next most frequent stochastic events were machine breakdowns, which occurred 18 times, and stochastic processing times, which occurred 13 times. Less frequently occurring stochastic elements in studies are machine additions [66] and stochastic transport times between operations [108]. The numbers are summarized in Table 7. Overall, as also observed for dynamic problem settings, environment behaviors are becoming more stochastic over time, posing more difficult learning problems but arguably modeling real-world-like scenarios more realistically in many cases.

5.2.4. Analysis of optimization criteria

By far the most frequently targeted optimization criteria are the makespan (also referred to as completion time) and the tardiness,

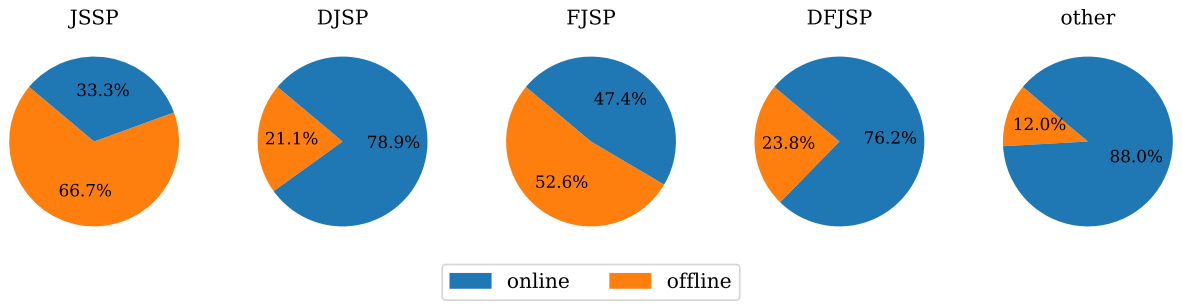


Fig. 7. Share of online and offline scheduling schemes per problem class.

Table 7
Sources of stochastic behavior and how often they appear in studies.

Source of stochasticity	Occurrences
Stochastic job events	37
Machine breakdowns	18
Stochastic processing times	13
Machine additions	1
Stochastic transport times	1

defined as the sum over all jobs of the difference in the jobs' completion time and their due dates. The makespan was optimized in 100/143 (69.9%) of all studies and the tardiness in 30/143 (21.0%) of all studies. Some studies optimize for other makespan-related objectives, like the machine utilization or throughput. However, many other interesting objectives were also identified: objectives related to reliability [109,110], such as the minimization of constraint violations [111], or urgency minimization [112,113]; capacity-related objectives like inventory levels [114] and waiting times in buffers [115]; sustainability-related objectives like the energy consumption [116–118], waste-material production [119], or carbon emissions [120]; financial objectives like electricity prices [121], production cost [109,122,123] or overall profit [124]; and customer satisfaction [125] or earliness of jobs [11,126]. It should be noted that not every objective is equally challenging to optimize for, since some can be evaluated more directly, like the machine utilization, whereas others are inherently delayed, like the tardiness. Also, there are many studies that evaluate multiple optimization objectives above-mentioned criteria (36/143, 25.2%), most often a combination of the makespan and the tardiness.

5.2.5. Analysis of online vs. Offline scheduling

We found that 85/143 (59.4%) of all papers describe online training schemes, implying that the agent is not given full information on the whole instance at all times, whereas the rest are offline scheduling schemes. Fig. 7 shows the relative shares of online and offline scheduling schemes split into the problem classes. It can be seen that dynamic problems are much more often addressed with online schemes. This is intuitive, because online schemes are purely reactive and can therefore handle stochastic events that are often present in dynamic problem settings by design. In static problems, offline schemes are more often used, since online scheduling can limit the action autonomy of the agent (cf. Section 5.3.2 on 'action logic' for more details of the action autonomy). It should also be mentioned that the best results on static instances are often achieved in offline scheduling schemes for this reason.

5.3. Analysis of the DRL design decisions

5.3.1. Analysis of single- vs. Multi-agent systems

Overall, roughly 76% DRL-based scheduling approaches make use of a single-agent system, as can be seen in Fig. 8. The right side of the figure shows the counts of single-agent and multi-agent systems in each year. Whereas the number of single-agent systems steadily increased over the years, the number of multi-agent systems fluctuates. However, we observed that the reasons why multi-agent systems are used have shifted over time. Generally, we identified four main reasons for the usage of multi-agent systems.

First, the notion that multi-agent systems have the potential to scale better to larger problem sizes, because one can simply clone agents that represent jobs or machines in the factory. Over time, this reason has lost its importance, as more powerful size-agnostic neural network architectures demonstrate effective scalability through the ability to generalize. The main difference between using an agent representing, for example, a machine and an agent acting globally is the reduced observation space, reduced action space, or local reward. Machine (or job) agents may only receive a local observation and act within their periphery (e.g., their buffer), learning about the consequence of their action on local metrics (e.g., their own throughput rate). Such an agent may be easier and more efficient to train and be applicable for different problem sizes, but can theoretically not reach optimal behavior for the whole system due to limited information. Once steps are taken in the direction of global information through a global reward or global observation, the problem could also be reformulated as a single-agent problem. Hence, deciding to build a multi-agent system for scalability boils down to a decision in the tradeoff between training efficiency and achievable optimality.

A second reasoning for using multi-agent systems is the natural fit of splitting the job- and machine-selection decisions needed for a single composite scheduling action into two distinct tasks for two agents. This reason is frequently encountered, which is reflected in the relative numbers for flexible scheduling problems (FJSP, DFJSP), where 38% use multi-agent systems, compared to only 24% in non-flexible scheduling problems (JSSP, DJSP). Here, multi-agent systems remain relevant and produce good results. However, alternatives to multi-agent systems for flexible problems do exist. These will further be discussed in the section on the analysis of action spaces (5.3.2).

The third reason is the implementation of a hierarchical RL setting, in which one agent steers the behavior of the second by defining different goals or rewards. Fourth, multi-agents are necessary where very different kinds of tasks are carried out, such as determining the release rate of new jobs into a system and scheduling. The third and forth reasons are very specific to the respective problem addressed or solution method devised, making it difficult to understand if there exist good single-agent alternatives without dedicated experiments comparing single- and multi-agent systems. Unfortunately, we found no such comparisons.

5.3.2. Analysis of action spaces

Action Validity The majority (86.7%) of approaches in the literature enforce action validity, as shown in Fig. 9. When plotted over time, as shown on the right side of Fig. 9, we observe a trend of diminishing cases in which valid actions are learned, while the number of publications in which valid actions are enforced is steadily growing. This trend is likely to continue because learning the validity of actions

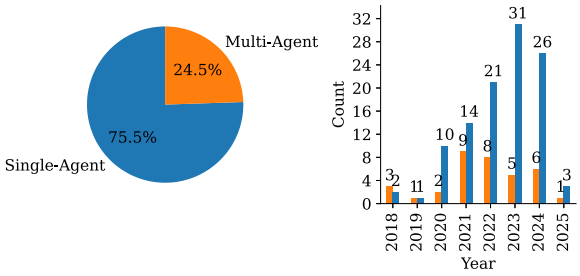


Fig. 8. Share of single- and multi-agent approaches overall (left) and over time (right).

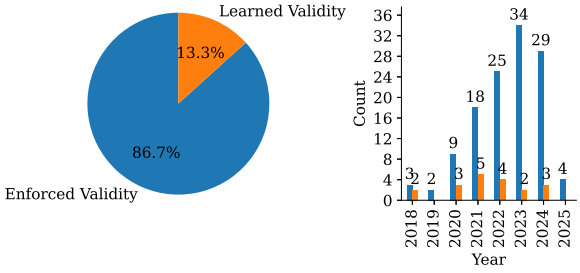


Fig. 9. Share of enforced and learned valid actions overall (left) and over time (right).

comes with certain practical drawbacks. From the learning problem perspective, having to learn which actions are valid is an additional task learned by the agent. The necessary punishment through high negative rewards introduces noise in the reward signal, making the learning task more difficult for the agent and complicating the reward engineering process for the algorithm designer. From a practical perspective, an agent that has only learned which actions are valid cannot be trusted to always suggest or execute valid actions during usage, which is a clear disadvantage. In theory, action validity could be used as a preliminary task to schedule optimization that could be used to help an agent to gain an understanding of the underlying problem during learning, before transitioning into a learning phase where the actual objective is optimized. However, we see no evidence in the literature that suggests that learning action validity is beneficial. Hence, we recommend enforcing valid action during both training and execution where possible. That said, there exist exemptions where action masking does not guarantee feasible solutions. For example, given time constraints between finishing an operation of a job and starting the next operations may lead to constraint violations that cannot simply be masked.

There are several ways in which the validity of actions is enforced in the literature. One common way is action masking, a technique in which the output values of value or policy networks are overwritten with $-\infty$ for those output neurons which represent invalid actions. It has been shown that this technique can be applied during training and still induce convergence [127], making it a straightforward and effective way to ensure the validity of actions as long as they can be determined by a set of rules. Another way to ensure that only valid actions can be chosen for value-based DRL approaches is to separately query the network for each feasible option. For example, when deciding between different machines, one can design an observation vector that contains unique information about each machine. The Q-network is then used to independently evaluate all valid actions (machines), thereby eliminating invalid actions as potential candidates [128].

Many authors circumvent the problem of invalid actions as a byproduct of their action design. All indirect action designs involve an interpretation of agent outputs as a set of rules which can easily mask out invalid decisions, effectively eliminating the existence of invalid actions in the action set of the agent. For example, when the action space consists of prioritization rules, each prioritization is always feasible since the chosen prioritization can be applied under the exclusion of invalid choices.

Direct vs. Indirect Actions 68,3% of all action designs make use of a direct action paradigm, giving the agent full autonomy over job or resource selection decisions or both. Fig. 10 discriminates between the two action logics as well as studies that feature a direct and an indirect action. One case, in which the action logic was unknown, is excluded from this figure. Heo et al. [129] make a direct decision between job types, but the prioritization between jobs within the chosen job type is done by the environment. It was classified as direct action. Note that we observe no clear trend in the overall ratio of direct to indirect action designs over time. However, as can be clearly seen on the right side of Fig. 10, indirect action designs seem to be much more prominent in dynamic problem settings. We assume that this is because such problem settings often pose a more challenging learning task for the agent and indirect actions, especially choosing between prioritization rules, guarantee a minimum of performance of the agent as well as easily obtainable strategy improvements by limiting decisions to sensible ones and eliminating invalid actions, as discussed before.

Single- vs. Multi-Decisions In 79.0% of investigated DRL designs, the agent takes single decisions in each decision-making step, such as choosing a resource, job, or rule to define a task allocation. Only 21.0% take multi-decisions, most often performing the simultaneous choice of a job and resource to process that job. As such, this design element naturally correlates strongly with the problem type. The JSSP and similar variants are often associated with single-decision designs, and the FJSP and similar variants are associated with multi-decision designs. However, there are exceptions to this trend. One example can be seen in [130], where a hierarchical DRL approach is chosen and one agent decides on a man-scheduling agent, which in turn adds arcs in a graph representation that are defined through a starting point (decision one) and an end point (decision two). Another exception for the JSSP is the work by Bonetta et al. [131], where a pointer network produces a whole schedule as one action consisting of multiple decisions.

Even more exceptions are seen for the FJSP and similar variants. Often, single-decision designs are realized through heuristic decisions on the resource, if the agent chooses a job, or a job, if the agent chooses a resource. Thereby, the agency of the agent is limited to one decision per step. But even where full agency is granted, single-decision actions are possible. More agency is granted if an agent chooses a composite indirect action, e.g., a prioritization rule, and full agency is granted if an action represents the choice of a job-resource pair. Alternatively, the decisions of the FJSP can be taken by two different designated agents in a multi-agent design, either in parallel or, more frequently, in sequence. On the neural network level, the multi-action designs can be implemented in different ways. One way is to have a multiclass-classification problem, i.e., some output neurons evaluate job actions whereas other output neurons evaluate resource actions and both are performed in parallel. Another way is to represent the hierarchy of decisions in the architecture. For example, if the job is chosen first in an output layer of the neural network, this output can be input to other layers which output the resource.

Interestingly, we found no publication which directly compares the implications and results of single- vs. multi-decision designs for problems which require two logical types of decisions per step, such as the FJSP. However, we can infer some practical advantages and disadvantages: The advantages of single-decisions are that they are easily implemented as composite decisions and that they require no imposed hierarchy of decisions. The main disadvantage is that they scale poorly, because the number of available decisions is multiplied. For example, choosing between ten resources and ten jobs results in an action space size of 100, whereas the multi-decision framework scales linearly with the sum of the number of resources and jobs. At the same time, the number of valid actions does not grow quadratically, such that the action space becomes increasingly sparse, which can hinder

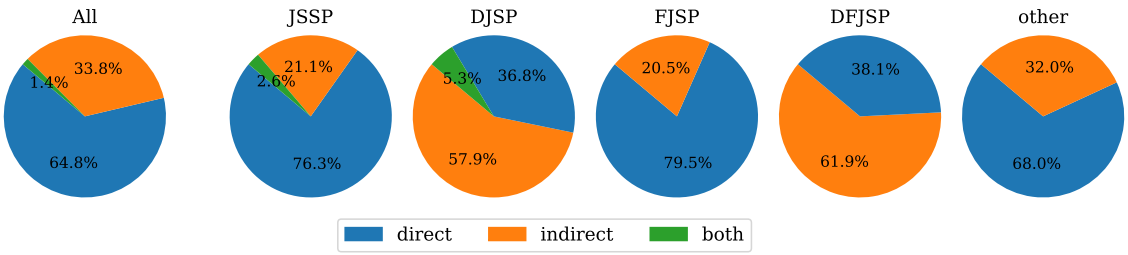


Fig. 10. Share of direct and indirect action spaces overall (left) and per scheduling problem type (right).

training effectiveness. One study by Kuhnle et al. [114] compares two alternative action space definitions on a JSSP variant. One definition allows the agent to suggest the next machine on which a job should start, the other defines a source and target machine for the next job to be processed. Since many source-target combinations are infeasible and invalid actions are only learned through rewards, the comparison suggests that the former, single-decision definition is faster to train and leads to better results. However, it is unclear if this insight is valuable in other problem and action definition constellations.

Action Logic The counts of applied action logics are depicted in 11, grouped by action logics belonging to time-based scheduling schemes (top) and those that belong to iteration-based scheduling schemes (bottom). One publication with an unknown action logic was omitted from this analysis. By far the most common action logic overall with an occurrence of 72.5% is the 'action-per-timestep' logic. It is followed by the offline logics 'Tetris-like' (15.5%) and 'adding arcs' (5.6%). The popularity of the 'action-per-timestep' action logic is likely due to four major advantages that arise from the time-based nature of the action logic. First and foremost, it is the most intuitive way to move along a timeline and always take scheduling decisions when decisions can be made. This property is shared with the action logics 'action-per-timestep, semiautomated', 'determine pr for whole schedule', and 'order dispatch and production sequence', although the latter two essentially take a decision in the beginning that makes further future scheduling decisions unnecessary. The second advantage is that the 'action-per-timestep' logic is compatible with discrete event simulations by design, facilitating the usage of established simulation frameworks and software as RL environments. Thirdly, it is applicable in online decision-making settings and for problem types that require reactive online decisions, especially in problem settings with dynamic or stochastic changes over time. This is an important advantage, since dynamic and stochastic settings are an integral motivation for the DRL-based scheduling community to use DRL because it has shown potential in video games under imperfect information and stochasticity [18,55]. A fourth advantage is that, compared to the 'Tetris-like' action logic, agents are biased towards scheduling operations as early as they could be scheduled. Therefore, random actions will lead to better results with this action logic than the 'Tetris-like' one. On the other hand, this bias restricts the agent's autonomy to theoretically explore the whole possible solutions space. This effect can only be overcome through the introduction of 'wait' actions. Those, on the other hand, can be difficult to evaluate through dense rewards and lead to more required actions per problem instance than 'Tetris-like' action logics, making the agent training more difficult. Another effect of the time-based action logic is that the whole simulation has to be stopped more often to check for applicable decisions, making the environments much slower than 'Tetris-like' ones. Overall, it seems as if the 'Tetris-like' and 'adding arcs' action logics result in better results in the vanilla JSSP, where five out of the best six best-performing algorithms deploy iteration-based logics. However, all iteration-based action logics require rescheduling, if dynamic events occur, in which case time-based scheduling is more efficient. The application of iteration-based scheduling techniques to dynamic problem remains an underexplored research area.

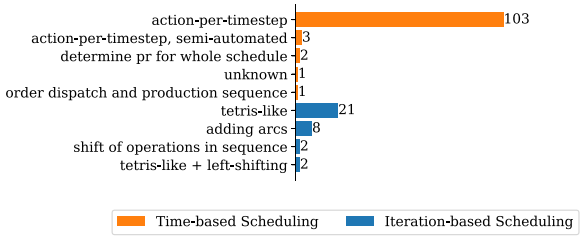


Fig. 11. Frequency of applied action logics in the literature corpus.

Compared to the 'Tetris-like' action logic, the 'Tetris-like + left shifting' variation takes away some of the mentioned autonomy to explore the whole solution space because scheduled operations may be shifted after they have been placed into the schedule to optimize actions algorithmically in retrospect. Although Zhang et al. [128] were successful with this logic, it has been observed that simple priority dispatching rules applied to this logic yield similar results [57], hinting at the bias this logic entails. The 'shift of operations in sequence' logic is less intuitive compared to the other logics regarding typical decision-making procedures and has, so far, only been used by a few researchers.

Action Space Size As stated before, the action space size is not suitable for an in depth statistical analysis due to the great variety of actions and according action space designs. We believe that the full table we provide online is most valuable to the reader interested in a particular publication to understand the action space more easily. Nevertheless, certain observations stand out. One observation is that most often, the action space size is related to the number of jobs or the number of machines in the problem setup. In 16 cases, the action space size equals the number of jobs, because the agent chooses from a set of next operations per job to process next. One additional action that represents a 'wait' or 'skip' action is also common. In indirect action designs, the number of priority rules to choose from ranges from three to 18 with an average around six that strikes a balance between enough options and the exclusion of ineffective prioritization rules.

5.3.3. *Analysis of observation spaces*

The number of different features that can be found in the literature is large. The most typical information includes: descriptions of operations, such as processing times and required resources; job level descriptions that build the statistical features of all operations belonging to the job, such as the sum, average, or standard deviation of processing times, or the ratio of already completed and remaining operations; resource level descriptions on the current and past utilization, current operations in the buffer, remaining work that needs to be done on them etc. Sometimes, this information is also combined through simple mathematical operations, e.g., by building the products and ratios of more basic features. More elaborate problem settings often require additional descriptive features about locations, priorities, deadlines, or similar aspects. The following is intended to provide only a rough distinction, since a more detailed description of every observation design would exceed the scope of this survey.

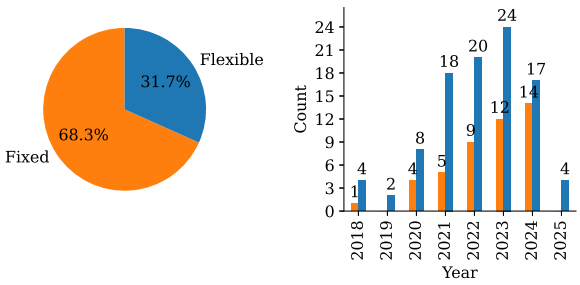


Fig. 12. Share of fixed and flexible observation sizes (left) and respective counts of occurrences over time (right).

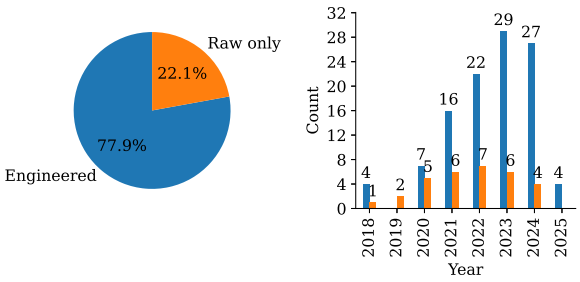


Fig. 13. Share of raw and engineered features in observation designs (left) and respective counts of occurrences over time (right).

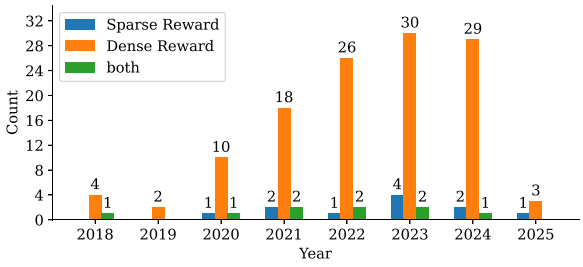


Fig. 14. Frequency of dense and sparse reward designs over time.

Fixed vs. Flexible Observation Sizes The first design element we analyze with respect to the observation space is the decision between a fixed and a flexible observation space size. As mentioned before, this decision also implies that a neural network architecture with flexible input dimensions is utilized, such as an RNN or GNN. Of course, this also impacts the generalization ability of a trained agent to be applicable to multiple problem sizes. Fig. 12 reports the share of fixed and flexible observation space sizes found in the literature as well as the counts how often they were used over the years. Roughly two thirds (68.3%) of all publications make use of an observation design of fixed size for one agent. However, the number of publications with flexible observation designs is growing faster than that of fixed observation designs, making up for half of all designs in 2024. We attribute the growing popularity to two trends. First, the trend correlates with a growing popularity of GNN architectures which facilitate flexible input sizes. Second, multiple studies have demonstrated that training on small problem instances and applying the trained network to larger problem instances can lead to state-of-the-art results while limiting the required computational burden of training on large problem instances [4,132–134].

Raw vs. Engineered Features Another analyzed design element lies in the usage of engineered versus raw features to form the observation. The relative share of both decisions as well as the counts across the years are shown in Fig. 13. Engineered features are used in 77.9% of all studies as opposed to the 22.1% of studies that rely solely on raw features. Looking at the trend of using raw features only, one can clearly see that the absolute number of these studies grew until 2022 and has since been declining. The growing number of raw feature observation designs until 2022 is most likely connected to the rise of more expressive neural network designs, such as GNNs, RNNs and self-attention based models, which through their structure can extract information from raw features more easily. The hope that raw features are interpretable enough and no feature engineering is required is further nourished by the vision of end-to-end DRL algorithms with multifaceted inputs and meaningful KPIs as output, as expressed in [18]. However, we observed that the algorithms with the best reported results combine raw input features and carefully engineered features. This realization can explain the gradual decrease in algorithms that rely solely on raw input features.

Myopic Observations Myopic observations are one of several manifestations of providing limited information to an agent. Generally, authors find a balance between providing full information on the current scheduling task and an intermediate solution to give the agent a maximal basis for decisions, and intentionally providing limited information to make the learning task easier for the agent. Certain designs provide only very limited information and are still successful. For example, Lee et al. [135] only include local information on a single machine in the system and only the number of waivers in the buffer of this machine along with the ratio of all completed jobs in the system. Ruichong and Fangyu [136] only describe jobs by their waiting times at their current machine buffer. Myopic observations can still provide enough information to make decisions. For example, the Shortest Processing Time First (SPT) priority rule relies on only information on the next operation per job. We found that 25 out of 140 publications (17.9%, three publications were excluded due to insufficient information) of publications only provide myopic observations to the agent. The number of such publications per year stagnates between four and six publications between 2020 and 2024, whereas the number of publications with a non-myopic observation rapidly increased. A reasonable assumption is that the decline of the relative share of myopic observation spaces is attributable to better overall techniques over time to train agents with more nuanced information.

5.3.4. Analysis of reward designs

The reward design presents a design decision that is tightly linked to the objective of each scheduling problem. The vast majority of reward designs follow a dense reward or a dense and sparse reward combination (131/142, 92.2%, one study excluded due to insufficient information). During the literature review, we found that many authors argue that dense rewards are generally regarded as leading to more stable convergence and better final performance for scheduling problems than sparse rewards. The most common dense reward in time-based scheduling settings is giving positive feedback proportional to the current machine utilization. The most common dense reward in iteration-based scheduling settings is giving a negative feedback proportional to the increase in makespan due to a scheduling decision. However, looking at the share of dense and sparse rewards over time, as shown in Fig. 14, reveals that a small number of studies maintain sparse reward schemes.

The small number of proponents of sparse rewards allows us to examine some individual argumentations for using them. Pirnay et al. [137] propose a self-competition based training approach, in which the superior policy receives a sparse positive reward for ‘winning’. In a competition-based setting, this reward strategy seems very reasonable and follows well-known pioneering work in board games [alphago]. Rinciog et al. [119] also take inspiration from such work and include Monte Carlo tree search, which mitigates potential downfalls of sparse rewards through forward simulations. The incentive to use sparse rewards for Su et al. in two studies [6,138] is that their evolution strategy is based on an algorithm specifically designed for sparse rewards.

A common reason for using sparse rewards is to solve multi-objective optimization problems, which balance multiple objectives with a composite reward [63,119,139]. Li et al. [140] use curriculum learning across two training stages, transitioning from a setup that punishes dense rewards to punishing total tardiness as a sparse reward. Unfortunately, they do not provide sufficient results to prove the necessity of sparse rewards.

Liu et al. [141] argue that stochastic machine breakdowns make evaluations of actions only possible in retrospect, i.e., through sparse rewards. However, this argument seems to be true for the deterministic scheduling case as well, where dense rewards work well. Burggräf et al. [142] design a sparse binary reward, evaluating whether or not a schedule leads exceeding a threshold throughput rate or not. Other studies provide no specific arguments for using sparse rewards. Of course, there is no reason to provide explicit reasoning for every design decision, especially since their designs seem to work well [10,134,143–146].

Three studies in our literature corpus directly compare the effect of a dense and a sparse reward design [114,147,148].

Pol et al. [147] examine two different reward designs for a multi-agent FJSP setting. One is a dense reward strategy that rewards local achievements of agents first and subsequently gives global rewards. In this reward strategy, the validity of actions needs to be learned. The second strategy returns a global sparse reward at the end of an episode in which invalid actions were masked by masking their Q-values. The results indicate that sparse rewards are favorable. However, the different approaches to action validity hinder a fair comparison. Additionally, these reward designs are intended for training multi-agent systems, so the results may not be applicable to other settings. For these reasons, we would argue that this study is not a strong indication that sparse rewards are generally superior in DRL-based machine scheduling.

Kuhnle et al. [114] tackle a real-world inspired machine scheduling problem with DRL and study both dense and sparse reward designs. In the dense case, a constant reward for orders entering or leaving the shop floor is given together with a linear combination of machine utilization and job weighting time related dense rewards. The sparse reward accumulates all these rewards in a buffer and returns zeros until the last step of the episode, where the accumulated sum is given. The authors conclude that the constant dense reward system works best. However, as before, since a very specific problem was analyzed, the transferability of this insight to other settings is, at least, questionable.

Samsonov et al. [148] compare three different reward designs on the JSSP. Their first, a dense reward, considers the relative difference of the intermediate makespan before and after an action. The second, a sparse reward, punishes the difference between the achieved makespan at the end of an episode C_{max} and the optimal makespan $C_{max,opt}$ of an instance in an exponential term, $r = 1000 \times 1.025^{C_{max,opt}-C_{max}}$. A third, self-proposed sparse reward compares the performance of the agent to that of itself in a previous iteration. The reward is positive if the achieved makespan is better than that achieved by the previous policy. It is negative if the achieved makespan is worse. For equal makespans, the reward is randomly either positive or negative. Their results indicate that the exponential sparse reward performs worst, the dense reward achieves intermediate results, and the self-proposed sparse reward performs best. Unfortunately, this does not answer if sparse or dense rewards are generally favorable but instead suggests that the details of the reward signal are most important. However, it should be noted that the experiments were conducted on rather small problem instances (up to 10×10 JSSP instances) and the sparse rewards are likely to become less effective for larger problem instances with even sparser signals.

Overall, the overwhelming majority of studies favors dense rewards, which we interpret as a signal that dense rewards are at least a good starting point for exploring reward designs.

5.3.5. Analysis of neural network architectures

The introduction of more efficient and more expressive neural network architecture elements, like residual connections or self-attention, has led to multiple breakthroughs in deep learning over time. It is therefore not surprising that new architectures are introduced as main contributions in many of the papers we analyzed. Fig. 15 shows how often distinct architectural features were used over the years. Note that in cases where there is insufficient information for clear classification, we assumed an FCN architecture and marked it with the FCN (?) label. It can be seen that the large majority of studies until 2022 used FCN architectures and have continued to be used by many, although a declining trend is clearly notable in favor of other architectures. There are two explanations for the long-lasting popularity of FCN: First, FCNs are the default option of many popular repositories for DRL algorithm implementations, like StableBaslines [149], and also reported to be robustly trainable in these libraries. Secondly, according to the trends in the observation space designs, engineered features are a more straightforward and often successful option, in which much of the structure of the observation is neglectable. However, all other architectures make use of the structure of scheduling problems by design. Note that it is hard to determine whether the introduction of more advanced architectures drove the trend toward fewer engineered observation features or vice versa.

Another very clear trend is the steady growth of Graph-NN architectures since 2020. Graph-NN architectures made up the largest share of all architectures in 2024. They are a natural choice for representing scheduling problems and states, as they capture the structure of scheduling problems, including precedence and no overlap constraints, by design. CNN and RNN architectures have been used throughout the years, but no clear trend is visible. Attention-based (ATTN) architectures, on the other hand, have clearly gained popularity. One should also recall that we counted graph attention networks as Graph-NNs rather than ATTN architectures. Apart from capturing the structure of the problem, all but FCN architectures have the additional advantage of providing the possibility to make agents problem-size agnostic, i.e., such agents can be trained on a certain problem size and be applied to another. Such techniques, especially leveraging shorter training times on smaller problem instances and generalizing to larger ones, have shown great success in recent years [128,133].

More characteristics of neural network architectures and how they relate to scheduling problems become apparent when grouping the usage of all architectures by problem type, as done in Fig. 16. The most striking observation is that RNNs are not used in dynamic scheduling variants. There is no clear explanation for this observation. One possibility is that RNNs are typically used for pointer networks or to read in a whole scheduling problem sequentially. These applications make them better suited for static problems, where the input size does not change, and less well suited when fast reaction times are required, which is often a focus in dynamic problems.

5.3.6. Analysis of DRL algorithms

In the analysis of used DRL algorithms we mainly distinguish between value-based and policy-gradient-based algorithms. How often these algorithms are used depicted in Fig. 17. Note that we excluded two multi-agent cases, in which one policy-gradient-based and one value-based agent were used in this figure [150,151]. The ‘Other’ entries in Fig. 17 summarize algorithms that were only used by in one study. For policy-gradient-based algorithms, these comprise Option-Critic, TRPO and unspecified other policy gradient methods. For value-based algorithms, ‘Other’ comprises TD3, DDN, TD and IQN. Recall that in studies comparing different algorithms, we only report the best performing option and name the others in Section 5.4.2 on benchmark algorithms.

PPO seems to be the most popular choice of DRL algorithms, which we attribute to the many existing stable implementations online and its reportedly stable convergence behavior in comparison to many other

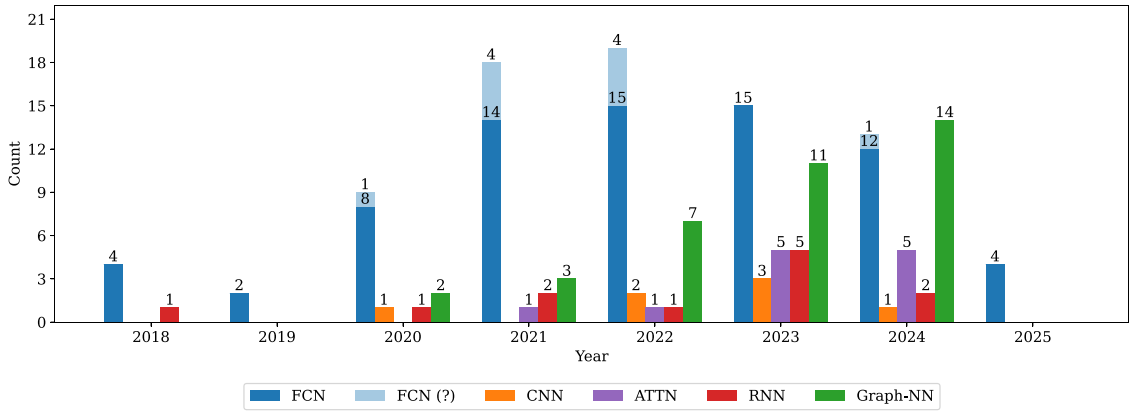


Fig. 15. Counts of the usage of neural network architectures over the years.

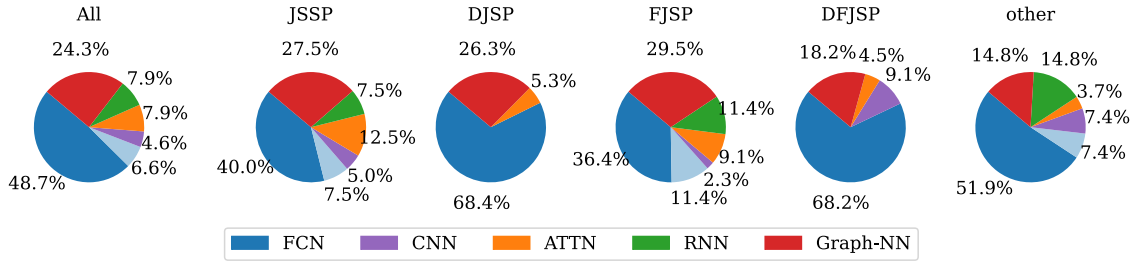


Fig. 16. Share of each neural network architecture usage overall (left) and per scheduling problem type (right).

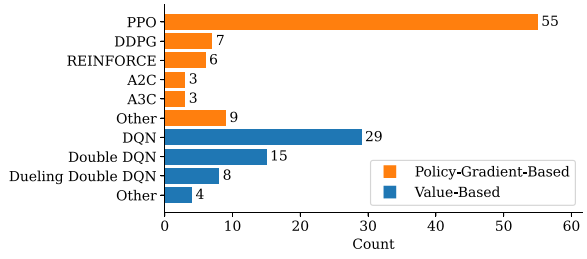


Fig. 17. Frequency of the classes of DRL algorithms used in the literature corpus.

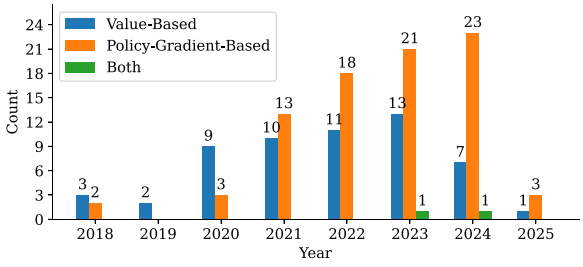


Fig. 18. Frequency of the classes of DRL algorithms over time.

algorithms. It is also applicable to both continuous and discrete action types, which makes it universally applicable. Unfortunately, with respect to reproducibility, the authors report the specific implementation of PPO only in rare cases, although very different implementations can be found online. The second most popular choice is DQN, followed by its advanced versions Double DQN and Dueling Double DQN. Although DQN often performs worse in direct comparison to more advanced algorithms, its popularity is likely due to the fact that it is easily implemented and pre-implemented in many software packages and libraries, including commercial software like MATLAB or ANYLOGIC. The most often observed technique that is additionally applied and algorithm-independent, is prioritized experience replay. Interestingly, almost none of the DRL algorithms used in scheduling are model-based algorithms, although the property of being able to plan ahead could yield great potential especially in stochastic scheduling problem settings. We believe that this field should be explored more actively as research progresses towards more stochastic, real-world settings.

Evidently, the two classes are relatively evenly distributed, with 39.9% of all studies applying value-based methods, 60.1% applying policy gradient methods and two studies applying both methods. Taking a closer look at how the distribution of the two classes changes over time in Fig. 18 reveals that the relative share of policy gradient-based methods has steadily increased in proportion to value-based methods that went from being less frequently used in 2018 to being used in the increasing majority of recent studies. This growing popularity of policy gradient-based methods is in line with our observation that policy gradient-based methods outperform the other class when directly compared in the same study.

5.4. Analysis of experimental setups

5.4.1. Analysis of benchmark datasets

Benchmark datasets are used in 60/143 (42.0%) studies to validate and compare the efficacy of proposed methods to that of other methods. The distribution of the usage of benchmark datasets is depicted in Fig. 19. Clearly, some benchmark datasets are more frequently used than others. For the JSSP, TA and LA are used most often, followed by ORB, FT and DMU. We observed that in the JSSP benchmarks, TA and DMU, and combinations of FT, LA, ORB, SWV, YN are often used within the same study. This is most likely due to the different problem sizes within the benchmarks, where TA and DMU contain larger problem sizes than the others.

The studies which use benchmark datasets are mostly congruent with those tackling standard scheduling problems, with some exceptions. For example, [152] build a method to tackle the DJSP with

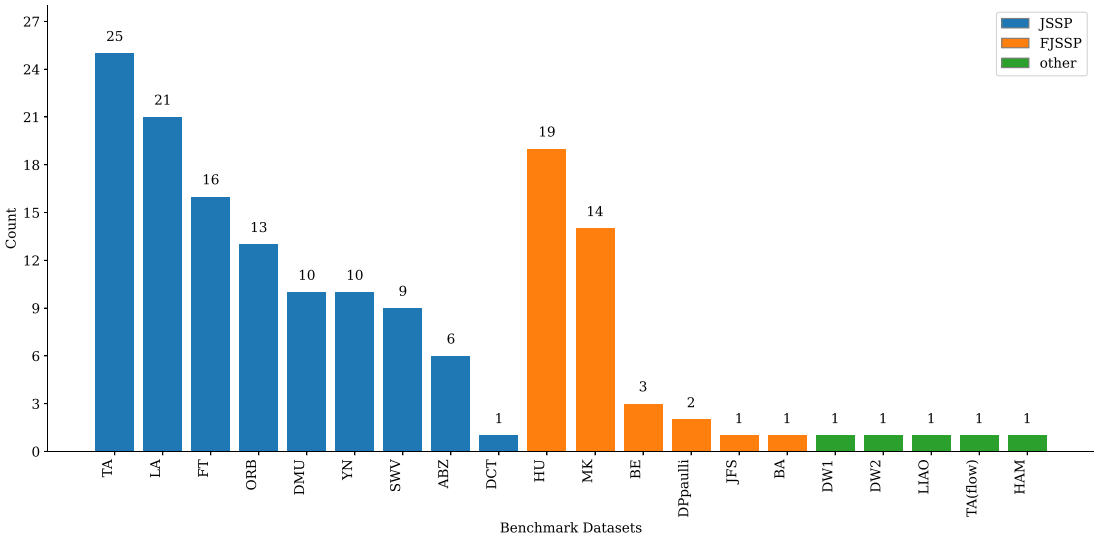


Fig. 19. Distribution of the usage of public benchmark datasets.

machine breakdowns and random job arrivals, but still test on the non-dynamic TA and DMU instances to showcase the transferability. In other examples, the authors tested on the TA benchmark but augmented the problem setting either with several available factories with identical machines [153] or with changing electricity prices for processing operations [121].

The benchmark datasets consist of only a few problem instances per problem size (1–10), likely due to the required computational effort to obtain optimal solutions. Although the small number of instances benefits large-scale experiments by limiting the required computational time, it leads to less trustworthy statistical outcomes. On a larger scale, it may also lead to algorithms biased toward the evaluation data. The negative aspects are further increased when studies only report results on a subset of instances from the benchmarks. However, when evaluations are performed on the entire benchmark data set, they are useful for comparing the performance of different algorithms between studies.

In order to analyze the progression of algorithms in the field, we look at reported results on benchmarks over time. We found 25 publications that test on the TA JSSP benchmark. However, upon closer inspection, only 15 out of 25 algorithm evaluations are performed on all problem sizes of the TA benchmark. In Fig. 20 the reported average makespans of one greedy policy rollout of these algorithms on the TA instances are plotted along the time axis corresponding to the publication date. One exception is the results by Tassel et al. [134], who only reported the best result among multiple stochastically sampled solutions. Generally, it can be seen that solutions for the JSSP have improved over time, with some outliers. However, the results must be compared with caution. The earliest result by Han and Yang [154], for example, only reports makespans on two problem instances per problem size to save computational time instead of reporting the makespans on all ten instances, which might explain the comparatively small average makespan. The same is true for [155,156]. In addition, in four cases the result could only be read off a plot and might not be very accurately represented here [4,6,156,157].

Overall, it seems that although the TA benchmark is useful, the way in which it is used is often suboptimal for the purpose of cross-study comparisons. As stated above, only 15 out of 23 studies evaluate their algorithm on all problem sizes. Only nine out of 23 studies report the average performance on all 80 problem instances. Many studies only evaluate their algorithms on chosen subsets, single instances, or only report averages across self-defined groups of ‘small-’ and ‘medium-sized’ instances. We strongly urge the authors of future studies to report results on all problem sizes and available instances to facilitate cross-study comparisons.

This is not the only limitation we observed. Algorithms are typically developed to strike a favorable trade-off between the achievable optimality gap and required computation time. From this perspective, in light of different hardware with parallelized processes, one should be aware that the reported makespans are only one element of the evaluation needed for the comparison between different studies and do not mean that one method is strictly superior to another one. The hardware-specific computation time is often not reported.

Although the reported evaluations on FJSP benchmarks suffer from the same limited comparability, we believe that a corresponding plot on the HU dataset may serve as a good starting point for researchers entering the field. The HU dataset, as discussed before, offers different flexibility levels, the r-data, e-data and v-data, which can cause confusion. We found that of the nineteen studies reporting to provide results on the HU dataset, many studies are not fully comparable. This is because they either report only results on one instance per problem size, use an unidentified subset of instances or do not report absolute numbers, but only how much better or worse other algorithms perform. Only seven out of the nineteen studies that report solutions on the HU dataset evaluate on all available subproblems. These and two others that report full results on one of the flexibility levels are represented in Fig. 21.

It should be noted that even within the remaining studies, reported optimality gaps are not fully comparable. Since the original publication by Hunrink et al. [67] only reports upper and lower bounds but not all guaranteed optima, researchers either report relative gaps to the upper bound [162] or to self-found upper bounds by using different versions and computation times of solvers [64,163], or do not report the considered optimal makespans at all. These might be reasons why there is no clear declining trend within the reported optimality of solutions over time, which we would have expected.

In conclusion, there are many public benchmarks of different types and sizes, which make algorithms more comparable. However, there is room for improvement in how these datasets are used. We believe the community would benefit from reports covering all benchmark data divided by problem size and potential subgroups. These reports should use metrics such as average makespan or optimality gap to evaluate performance. Additional notes on the required computation time and hardware used are also recommended, especially if the code is unavailable for reproduction on different hardware or software versions, as is currently the case in only 23 out of 143 publications.

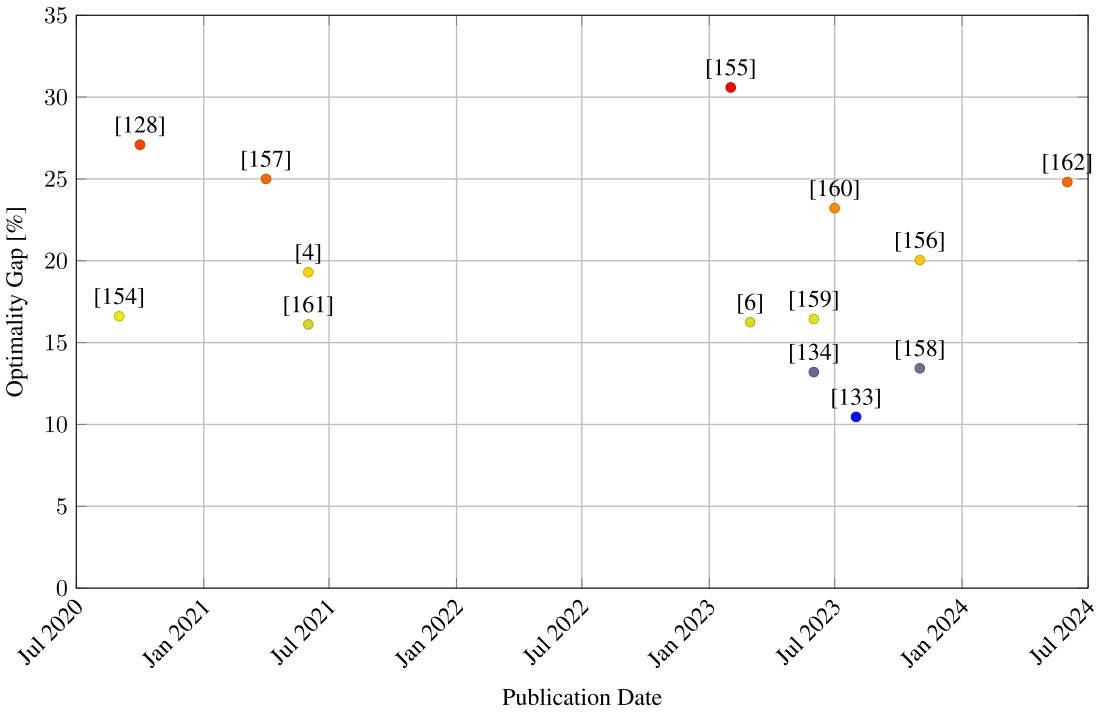


Fig. 20. Scatter plot of the AVG data with associated citations and values over time (see [158–161]).

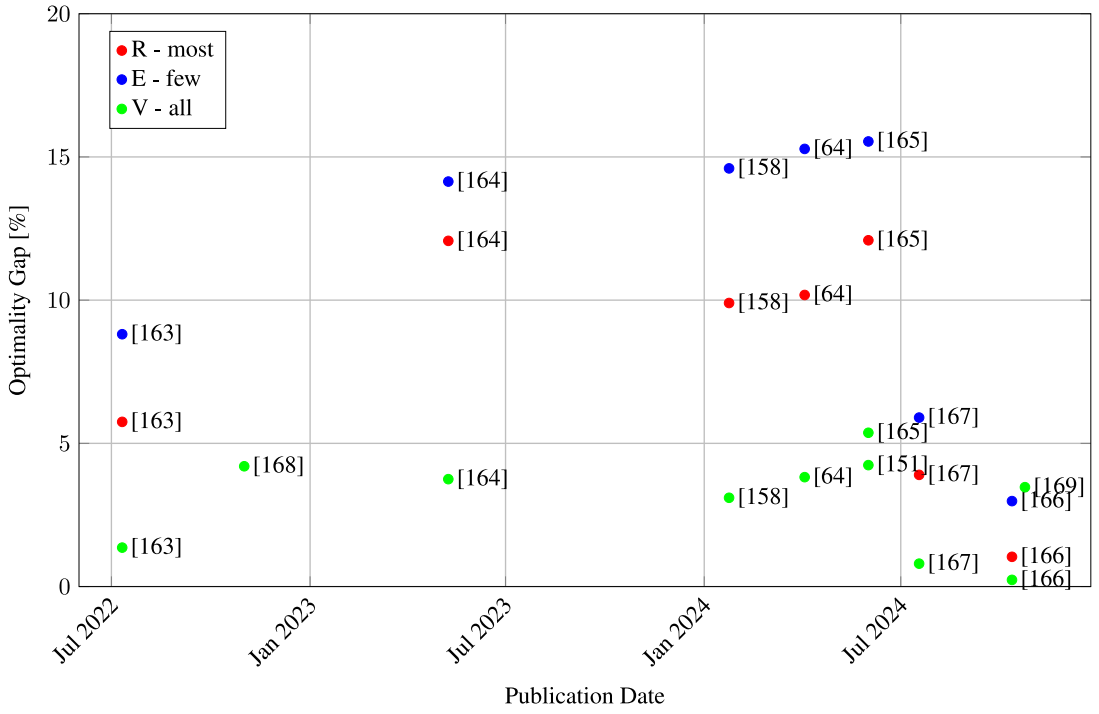


Fig. 21. Scatter plot of the data with categories R, E, and V over time.

5.4.2. Analysis of benchmark algorithms

Priority Rules Fig. 22 shows how often priority rules are used as benchmark algorithms in the literature for all rules with a count greater than four. The plot shows that it is very common to compare the performance to priority rules. In fact, 79.7% of all studies do so. Significantly, more than half (62.2% percent) of the studies compare their results with SPT and almost half (46.2%) compare with FIFO. This is interesting since SPT, FIFO and LPT are reactive and short-sighted, relying only on information of a single operation within a job. Other popular rules consider the sum of processing times or the number of operations, such as MWKR, MOR, LWRK and LOR, and are therefore less short-sighted. However, even these still neglect the distribution of future operations on resources.

Of course, not every rule is applicable to every problem. When due dates are provided, due-date and slack-time related rules, such as EDD and MST, are popular. In dynamic settings, the critical rate (CR) is also used for prioritization. Some authors also provide a comparison

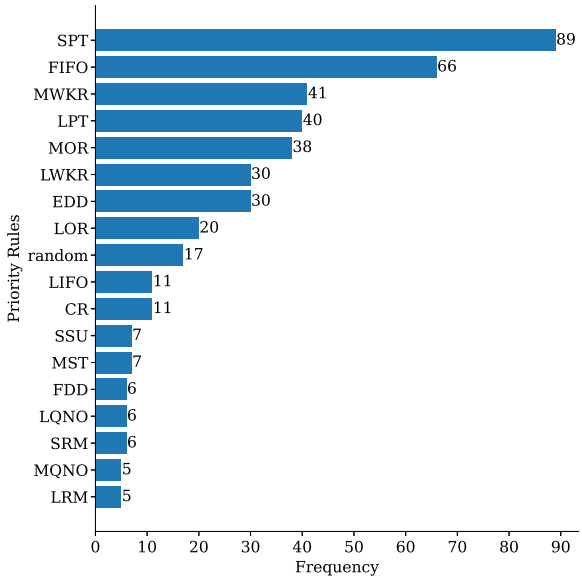


Fig. 22. Counts of used priority rules in the literature corpus.

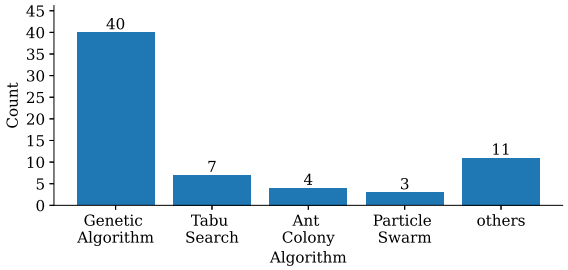


Fig. 23. Number of comparisons against different metaheuristic.

to a random policy to demonstrate that the DRL agent has learned something at all.

The overall impression of priority rule based benchmarks is that they provide a weak but intuitive relative baseline for each individual DRL algorithm. However, the rules seem less useful for evaluating the absolute performance of a DRL algorithm or for performance comparisons between different studies. This is not only because the priority rules typically perform 15%–35% worse than optimal, but also because the way in which the prioritization is used differs largely between papers depending on the action logic and the mathematical implementation of the rule. Hence, the same priority rule applied to the same benchmark datasets may yield very different results.

Unfortunately, such details are often not provided. We therefore argue that it is imperative to clearly state the mathematical formulation of each used priority rule as well as the pseudo-algorithm for how it is applied to solve a scheduling problem.

Despite these minor shortcomings, reporting the results of priority rules is beneficial for performance evaluation, especially given the difficulty of comparing studies based on performance on benchmark datasets, as described above. Priority rules and their performance on specific problems are valuable for deriving meaningful features and evaluating action logic. This serves as a good starting point for making informed design choices, especially when dealing with the multiple constraints and optimization criteria of real-world use cases. Therefore, we encourage readers to explore the supplementary online table, which lists the priority rules used as a benchmark in each study.

Metaheuristics The benchmark algorithms used for comparison show that metaheuristics were used in about a third (34%) of the total studies reviewed. Fig. 23 summarizes how often specific metaheuristics are used as benchmarks. The percentages shown are based on the 49 papers identified as using at least one metaheuristic benchmark. Genetic Algorithms (GA) and their variants were by far the most common choice, featured in 40 comparisons (62%). Tabu Search (TS) was the next most frequent (7 comparisons, 11%), followed by Ant Colony Optimization (AC, 4 comparisons, 6%) and Particle Swarm Optimization (PSO, 3 comparisons, 5%). Within these main categories presented in the table, different variants and specific implementations (e.g., different types of GAs or TS) are summarized under their primary metaheuristic category. Other less common metaheuristics accounted for 11 comparisons (17%). The total number of comparisons (65) is greater than the number of papers using metaheuristics (49), as several publications benchmarked against multiple different metaheuristics.

Comparisons between studies using metaheuristics and specifically Genetic Algorithms (GAs) as benchmarks face significant challenges. Even when referencing the same general variant (e.g., standard GA, two-stage GA), crucial implementation details often differ, impacting performance. A primary issue is the frequent omission of hyperparameter settings (such as population size, operator probabilities, or generation counts). As these are typically tuned for specific problems, their absence makes fair performance assessment speculative. Furthermore, variations in the implementation specifics (e.g., crossover points and mutation types in GA) and, fundamentally, the encoding/decoding schemes used, profoundly affect the search process and outcomes. These essential details vary widely and are often unreported. Consequently, without standardized reporting, direct 'apples-to-apples' comparisons of publications are often unreliable.

Solvers Around 20% of all studies use exact numerical solvers as benchmark algorithms. Of these, around 60% use a solver from the Google OR-Tools package [8] and 20% use Gurobi [7] or IBM's CPLEX software. Google OR-Tools provides an online tutorial for solving the JSSP in Python, granting easy access, and has been advocated by [76] for best performance on large problem instances, which explains its popularity in the community.

Since the solvers provide a guarantee of optimality, they may be used to find a hard baseline for the achieved objective functions of DRL algorithms. For certain standard problems, researchers treat the solvers as competitors and compare the trade-off between the achieved objective value and computation time. In these cases, solvers are typically time-constrained and, on large problem instances, do not return optimal solutions.

Comparison to exact solvers has limitations. For example, we found no study using exact solvers in dynamic problem settings. It can be argued that since not all information and no predictions of i.e. newly arriving jobs are available, exact solutions quickly become infeasible. However, in theory, one could still deploy solvers iteratively whenever new information is available. Another factor limiting the usefulness of solvers as benchmarks is that authors do not provide sufficient implementation details. The exact version, algorithm within the software suite and problem encoding have a large influence on the performance of the solvers. In general, Google OR-Tools users employ a CP-SAT solver whereas Gurobi and CPLEX users employ MILP solvers. However, such information is often missing.

Third-Party DRL Algorithms Comparisons with DRL algorithms developed by other researchers can demonstrate superiority over previous DRL-based methods in terms of better objective values or shorter computation times. The number of comparisons to each of the algorithms is listed in Table 8.

With 17 studies comparing their results to L2D [128], it is the most popular DRL-based benchmark on the JSSP, followed by ScheduleNet (5 comparisons), DDDQNPR (4 comparisons), and L2S (3 comparisons). The popularity of L2D is understandable, given it was one of the first breakthrough publications in the field in 2020, as it introduced the end-to-end approach with GNNs and has a corresponding code repository, making it easy to reuse and apply. However, we would

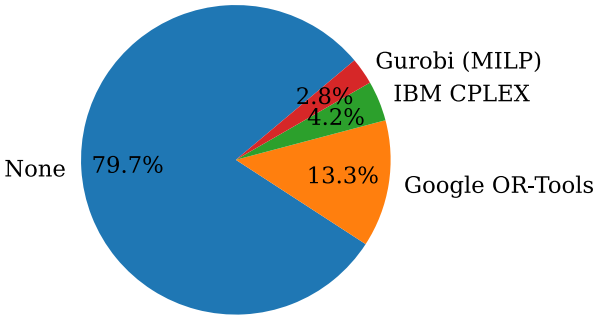


Fig. 24. Usage of exact numerical solvers in the literature.

Table 8
Overview of baseline DRL-based scheduling methods, their target problem types, number of comparisons, and references.

Name	Year	Designed for	# Comparisons	Ref.
L2D	2020	JSSP	17	[128]
DDDQNPR	2020	JSSP	5	[154]
ScheduleNet	2021	JSSP	4	[164]
L2S	2021	JSSP	3	[4]
DGERD	2023	JSSP	2	[155]
DRL-Liu	2020	JSSP	2	[10]
E2E-DRL	2021	JSSP	2	[165]
DRL-Wang	2021	JSSP	1	[59]
MCDQN	2019	JSSP	1	[166]
NeuRewriter	2018	JSSP	1	[167]
RS	2023	JSSP, FJSP	1	[168]
DRL-S	2022	FJSP	7	[163]
MPGN	2022	FJSP	4	[169]
DRL-Wang24	2024	FJSP	2	[170]
DRL-Yuan	2024	FJSP	2	[171]
DRL-Ech	2023	FJSP	1	[172]
DRL-Zeng	2022	FJSP	1	[173]
DRL-DU	2022	FJSP	1	[117]
DRL-Han	2021	FJSP	1	[174]
DRL-Wu	2024	DJSP	1	[175]
D3QPN	2022	DJSP	1	[65]
DRL-LuoBin	2021	DJSP	1	[176]
DRL-Luo20	2020	DFJSP	2	[177]
DRL-Zhang23	2023	DFJSP	1	[178]
DRL-Luo22	2022	DFJSP	1	[66]
DRL-Luo21	2021	DFJSP	1	[179]
PM-Li	2023	PM	1	[140]

argue that having been outperformed by relatively large margins numerous times, its relevance as a benchmark might decline in the future. This is underscored by the fact that more recent studies often include not just L2D but multiple third-party algorithms. On the FJSP, seven comparisons to DRL-S [163] make it the most popular DRL-based benchmark algorithm in that problem class. Like L2D, the corresponding code is available online, facilitating reproducibility and adaptability. For dynamic scheduling problems, there does not seem to be a clear DRL-based benchmark algorithm tailored to dynamic characteristics. Instead, construction heuristics from the non-dynamic setting are adopted. We attribute this circumstance to the field's youth and lack of established benchmark datasets, as well as the large diversity of studied use cases in the literature. This makes it more difficult to conceptually and programmatically integrate third-party solutions.

Own DRL-Variations Own RL-variations do not serve the purpose of making different studies comparable. Rather, own variants are an important part of the evaluation of own designs as they allow readers to understand the impact of single elements and design decisions or a proposed algorithm. Some variations are close to hyperparameter searches. For example, the most common variation is to switch DRL algorithms (PPO, DQN, A2C and others), which is often facilitated by used DRL frameworks. Less frequently, different neural network designs are compared, followed by different reward design choices. More elaborate variations are carried out when comparing proposed multi-agent approaches to centralized single-agent approaches. We argue that hyperparameter searches are always a good option to include in supplementary material, if available. More elaborate changes are useful when a specific design choice is central to a study and its superiority is best displayed when keeping as many variables fixed as possible and only changing the specific design choice. Such ablation studies are common practice in machine learning research and should be adopted in machine scheduling research, especially when artificial datasets are used.

5.4.3. Analysis of generalization ability

The generalization ability is an integral study design decision that also has a drastic impact on the comparability of reported algorithm performances. The literature is categorized into the four generalization ability grades we defined in Section 4.4, each with certain advantages and disadvantages.

In the first grade, an agent is trained on one or more instances in order to obtain solutions for them. In theory, this only requires that an agent can discriminate between the steps through the observation, such that it can explore different action sequences during training, to converge towards a good solution. This is advantageous, because the agent does not have to generalize but can overfit on a problem instance, which can lead to relatively fast convergence. This strategy is comparable to a search procedure or genetic algorithm, and therefore inherits the same unfavorable property that, compared to other generalization ability stages, many computations are required to solve a single instance. Compared to other search strategies, DRL-based search is often less efficient due to the many expensive neural network inferences and parameter updates needed during training. In the analyzed literature corpus, 41/143 (28.7%) of studies do not conduct train-test data splits. While the resulting test makespans are often better than the reported baselines, the required computational time is a major disadvantage. It is also questionable whether DRL is the best option among algorithms that do not produce generalizable strategies. Especially in practice, where algorithms may be used in an online fashion, it likely becomes challenging to fulfill computational time requirements. Either way, the results of the first generalization ability grade cannot be fairly compared to those of the other grades.

Algorithms of the second grade are typically trained on thousands of problem instances and tested for generalization on problem instances with the same number of machines, number of jobs, processing time distributions, and other characteristic properties of the training problem instances. Agents trained to perform well in this setting should avoid overfitting by finding individual solutions in the training dataset. Instead, they must learn a general strategy for finding solutions. This is advantageous because no additional training time is needed to generate solutions for new instances. Only neural network inferences and simulations are required. Since we treat this generalization grade as mutually distinct from the first grade, 102/143 (71.3%) of studies fall into this category.

Third grade generalization ability algorithms, which generalize across different and unseen problem sizes between training and testing, are employed in 51/143 (35.7%) of reviewed studies and build a subset of all second grade algorithms. Algorithms that generalize when trained on small problem instances and tested on large ones are advantageous. They require less computational time during training, are easier to monitor since computing optimal solutions for small instances through constraint programming is fast, and are flexible with respect to the problem setup in which they are ultimately deployed. It has been argued by Iklassov et al. [133] that training on multiple problem sizes yields better overall solution strategies than training on a single problem size. However, this has not been confirmed in other studies yet. If this assumption holds and the forced generalization ability

across problem sizes is not inferior to agents specializing in solving one problem size, it is fair to compare the performance of grade three algorithms with grade two and four algorithms.

19/143 (13.3%) of studies examine the fourth grade of generalization ability, in which the testing data differs from the training data in at least one characteristic other than and not necessarily including the problem size. Hence, grade four is not a subset of grade three. The most common changing characteristics are changing order arrival frequencies in dynamic job shops, machine capability distributions in the flexible job shop, and changing processing time distributions from which times are sampled. Such generalization ability is very useful in practice, where such settings are likely to change over time. All studies testified a very good generalization ability in this aspect, which underscores the large potential in DRL algorithms for machine scheduling problems and is in line with the motivation and long-term vision of the research community. However, none of the studies have conducted research specifically examining at what point out-of-distribution generalization fails. Such studies would be valuable in building trust in the algorithms and facilitating the transfer of technology from research to industry.

5.4.4. Analysis of code availability

The availability of code and trained model weights is a significant benefit for researchers aiming to understand a method better, to build on top of an existing method, or to compare their results to an existing method. In cases where the addressed scheduling problem types diverge from standard problems, code also facilitates research on these problems and can lead to faster progress on such problems over time. Yet, relatively few studies (23/143, 16%) publish their full code. Unfortunately, we also see no trend towards a higher percentage of code repositories that are made available over time. The only trend we observed was that there were relatively more repositories on the JSSP (12) than on the FJSP (5).

6. Critical discussion

6.1. Summary and discussion of the results

How can different DRL-based solutions be discriminated and which solution designs exists? In this survey, we covered publications on variants of the Job Shop Scheduling Problem (JSSP) and the Flexible Job Shop Scheduling Problem (FJSP), as well as their dynamic extensions (DJSP and DFJSP) solved with deep reinforcement learning (DRL). They additionally address other scheduling problems with various constraints and objectives. We introduce a framework that categorizes problem types, algorithmic design choices, and experimental setups. Using this framework, we evaluate the current state of the field, identify significant trends, and thereby provide a response to the first research question of this survey.

We found that research on DRL for scheduling problems has grown substantially, evidenced by a substantial increase in publications over recent years (see Fig. 4). This field is highly interdisciplinary, situated at the intersection of manufacturing, computer science, operations research, and artificial intelligence. As a result, relevant studies are published across a wide range of scientific outlets, with no single outlet yet established as the primary one for innovations in this area (see Fig. 5).

What types of machine scheduling problems are being researched? Our review reveals that the surveyed articles cover a broad spectrum of scheduling problems (see Section 5.2). While the large majority of studies are conducted on the JSSP, FJSP, DJSP, and DFJSP, there is a clear trend towards including a growing number of constraints and dynamic elements in the studied problem settings, which is attributable to the continuous progress in this area (see Fig. 6). However, real-world applications remain underrepresented, as academic problem settings continue to be of interest, especially to the part of the research community that primarily uses scheduling problems as a testbed for new algorithms. The research area would greatly benefit from representative field studies of real-world problems, where measurable results are published and rigorously compared to alternative solution methods. This should be a focus of future work. We also found many mismatches between stated problem objectives and reward designs used to achieve them, which makes the classification of problems challenging at first sight, which should be addressed in future studies.

What are promising starting points for design patterns when addressing a scheduling problem with DRL? And what are the implications of making certain design decisions? With respect to the third and forth research question, we observe the following: Regarding DRL algorithms, the majority of reviewed approaches rely on model-free DRL algorithms, most notably PPO (see Fig. 17). In terms of architectural design, graph neural networks and attention-based networks are gaining popularity due to their ability to capture the relational and sequential structure of scheduling problems by design (see Fig. 15). Despite their theoretical ability to represent the whole problem instance, results and trends suggest that the integration of carefully engineered features is still a major factor for the success of the design. Future studies should analyze the importance of engineered features in light of more expressive neural networks explicitly to substantiate this claim.

We also found evidence suggesting that the use of more powerful architectures and DRL algorithms has enabled the incorporation of broader observation spaces. These spaces integrate all available information on a problem instance, rather than limiting the information to local factors around a job or machine. As a consequence, the usage of multi-agent systems aiming at distributing the decision-making process to agents based on local information is declining in favor of agents with a global perspective (see Fig. 8). We further found that rewards are designed densely whenever possible (see Fig. 14). Action space designs are highly heterogeneous, varying because of adjustments that need to be made for different scheduling problem scenarios, but also within the same ones. We observed that generally, agents are granted less decision autonomy for problems with more stochasticity and constraints, while agents are granted more autonomy in more abstract problem settings that allow for finer-grained decisions. However, this trend may be temporary until better overall algorithms are developed.

What are the major trends and open research gaps in DRL-based machine scheduling? Beyond the above-mentioned general trends, identifying design patterns that consistently outperform others across different designs remains challenging (see Section 5.4). Therefore, the fifth research question could only partially be answered. This difficulty arises from two factors: first, many studies introduce multiple novel design elements simultaneously, without a dedicated analysis of the impact of single decisions, making it difficult to determine the success of an individual design element. Second, performance comparisons across studies remain challenging due to inconsistent generalization goals and the inconsistent use and implementation of benchmark algorithms and datasets. Future work on the establishment of a standardized testing suite featuring benchmark datasets and standard reporting schemes along with computation times would be a great addition to the research field for this reason. Overall, it should also be mentioned that the limitations current DRL-based solution approaches have compared to established methods are often not clearly communicated: successful designs still rely on manual designs and human expert knowledge; the solutions are often slower than established alternatives but their latency is not sufficiently compared; the uncertainty of agent actions remains underexplored. Such elements, too, should be the focus of future work.

6.2. Recommendations for future work

In this section, we interpret and distill our results to offer recommendations for approaching different research scenarios. We also

outline how this review may support researchers and practitioners across different stages of their work. While the recommendations are inherently subjective, they are grounded in our intensive engagement with the topic during the literature analysis.

At one end of the spectrum are industrial proof-of-concept studies aiming to demonstrate that DRL-based scheduling is a viable option for novel, previously unstudied problems. In such cases, instead of drawing inspiration from the best-performing models on the standard JSSP or FJSP, it is advisable to identify studies in which the problem description closely resembles the one to address. The tabular data we provide can serve as a useful starting point for exploring relevant prior work.

Subsequently, following a common practice in the literature, it can be helpful to view frequently chosen design decisions as implicit indicators of robustness and stable learning behavior. According to this rule of thumb, for any encountered problem setting a good starting point would be a single-agent approach with a dense reward design, a straightforward MLP network architecture, and PPO as the DRL algorithm. When dealing with a novel problem, it might be helpful to start by creating a corresponding environment. Here, a Tetris-like action logic should be preferred over an action-per-timestep approach, if applicable. Then, the environment should be tested with various prioritization rules.

This not only serves as a good baseline to interpret the performance of a trained DRL agent, but can also inform a suitable engineering of features that are the foundation of a particularly well-functioning priority rule. Since many studies report the superiority of letting an agent choose between different priority rules over applying a single one, an indirect action space that does so is a good starting point for any new problem setting. If the learning behavior is stable, a more sophisticated neural network architecture, direct action spaces, and greater levels of generalization ability are next steps to improve the algorithm's performance. As a first framework for the implementation, the *schlably* [180] repository can be a source of inspiration.

At the other end of the spectrum of studies investigated are advancements on algorithms for solving the standard JSSP and FJSP or their dynamic counterparts. In this context, concrete examples exist that describe successful and unsuccessful approaches, offering direct sources of inspiration. This survey provides a list of the most cited and, for static problems, currently best-performing algorithms, along with common benchmark algorithms. These results should be used with caution, since only few papers report every implementation detail (parameter counts, learning curves, inference latency, environment step speed, generalization results). In addition, incorporating identified trends can be highly beneficial when approaching any new problem setting. Examples include combining raw and engineered features, using GNNs and attention-based architectures, and training on small problem instances to generalize to larger ones. Places to search for trends from the AI community include the proceedings of AI venues, such as NeurIPS or ICLR, or the journal IEEE Transactions on Neural Networks and Learning Systems. A particularly inventive community that lies between OR research and AI is the Neural Combinatorial Optimization (NCO), which often aims at tackling multiple combinatorial problems, including scheduling problems, with the same methodology [106,167,181]. For easier comparison and as a source of implementation inspiration, we recommend using the RL4CO library by Berto et al. [182] and the benchmark repository by Reijnen et al. [183].

The final recommendation stems from the considerable effort required to extract the information used to categorize the reviewed studies. While it is not necessary to adopt the exact nomenclature and definitions introduced in this survey, we believe the characteristics analyzed in this study are of broad relevance to the community. We therefore suggest using these characteristics as a checklist when reporting on scheduling problems, methods, and results to ensure that the corresponding information is provided in clear and explicit statements. Our review revealed that critical details regarding the observation space, action logic, and generalization ability were frequently either underspecified or described in an ambiguous manner.

6.3. Limitations

The literature review conducted in this paper is not without limitations. One limitation lies in the scope of the literature corpus. Among other restrictions, standard flow shops were excluded from the review. This decision is based on the reasoning that standard flow shops present an easier but related machine scheduling problem compared to job shop problems and that, therefore, insights derived from the literature on job shops are more easily adaptable for flow shop problems than vice versa. In this regard, the analyzed corpus is biased towards job shop problems. However, we recognize the benefits of including literature on flow shops and will consider doing so in future work. Similarly, since our performance comparisons are based on studies on the same, limited benchmark datasets, our conclusions and recommendations may be biased towards certain problem sizes.

Another limitation of this review is its narrow focus on how DRL is applied to enhance scheduling algorithms. Motivated by the vision of end-to-end DRL-based solutions for machine scheduling problems presented in Section 1.1, we primarily analyzed literature in which DRL agents are granted a lot of control over the scheduling process through prioritization in simulation-based constructive optimization schemes. However, the application of DRL agents to replace stochastic or heuristic decisions in metaheuristics, such as genetic algorithms [98] or bee colony algorithms [184], was not part of the analysis. Such algorithms will have different action logics and reward systems, but share other design elements, such as observation spaces and NN architectures with constructive approaches, rendering parts of the algorithm designs useful for inspiration and adaptation.

Additionally, it is limited to DRL. Noteworthily, so-called self-labeling or self-improvement techniques were left out, in which multiple trajectories are sampled on the same instance and the best sequence of actions is used to generate classification labels, although have shown promising results on the standard JSSP [5,181]. Most of the design decisions that are discussed in the context of DRL-based solutions also apply to these supervised-learning solutions, with the notable exception of reward design. However, to the best of our knowledge, no studies have explored the application of these alternative techniques in stochastic or dynamic scheduling environments. As such, their generalizability across the broader range of scheduling problem settings remains an open question for future work.

Lastly, we have generally treated increasing trends as positive signals for the effectiveness of a design decision and decreasing trends as negative signals. However, it is important to acknowledge that such trends may not necessarily reflect true performance differences. Instead, they may be driven by external factors such as the availability of open-source code, ease of implementation, or prevailing trends in adjacent research fields. For instance, the growing adoption of attention-based neural network architectures may be influenced more by their success in natural language processing than by demonstrated effectiveness in scheduling contexts. Unfortunately, till now no ablation studies have been performed in this regard.

7. Summary

This literature review provides a comprehensive and structured overview of research at the intersection of deep reinforcement learning (DRL) and machine scheduling. By examining 143 studies, we identify the diversity of scheduling problem types addressed, ranging from academic benchmarks to more applied scenarios, and examine how DRL methods have been designed to fit these varying scenarios. Our work categorizes studies and their algorithm design choices according to key features such as observation and action space configurations, reward formulations, and neural architectures, revealing trends and sharing insights into how these elements influence the effectiveness and applicability of DRL approaches. Notably, we also showcase trends in design patterns emerging in response to specific scheduling scenarios. Our

Table A.9
Dispatching rules mentioned fewer than five times in the literature corpus.

Rule	Explanation	Synonyms
SPT	Shortest processing time	SSO, JSPT, SOT
FCFS	First Come First Serve	–
EDD	Earliest due date	ECT, JECT
Weighted SPT	Weighted shortest processing time first	–
TWK	Minimum total working time	SJF
EAT	Earliest arrival time	EJF
LAT	Latest arrival time	–
LW	Least weight	–
HW	Highest weight	–
SMWT	Smallest remaining waiting time	–
SCRJ	Smallest completion rate	–
NJF	Nearest job first	–
FAA	Fastest available machine accepts job	–
MMINU	Machine with minimum utilization	–
TC	Time constraint heuristic	–
JMINP	Job with minimum processing power (carbon emission metric)	–
JMAXP	Job with maximum processing power (carbon emission metric)	–
SCT	Shortest changeover time	–
EST	Earliest start time	–
JSTT	Job with shortest transportation time	–
JLTT	Job with longest transportation time	–
AVPRO	Smallest average processing time per operation	–
SIT	Smallest idle time	–
MAS_PAR	SPT with family setup rule	–
PAB	WIP-related rule (semiconductor setting)	–
EF	Earliest finish	–
SS	Static slack	–
LED	Largest expected delay	–
SRT	Smallest relaxation time	–
EDF	Earliest cutoff-time	–
BATCS	Batching apparent tardiness cost with setups	–
COVERT	Slack-related rule (Carrol, 1965)	–
ERT	Earliest release time	–
LS	Largest size	–
Setup-based	Semiconductor-specific rule	–
Plan-based	Semiconductor-specific rule	–
SJFS	Shortest Job First Search	–
ATC	Apparent tardiness cost	–
RPT× TRPT	Remaining proc. time × total remaining proc. time	–
RPT/TRPT	Remaining proc. time ÷ total remaining proc. time	–
RPT/OR	Remaining proc. time ÷ remaining operations	–
LWT	Least total proc. time left on machine	–
SPTM	Shortest processing time of machine	–
MWKR+	Most processing time remaining after next operation	–
MOD	Modified operational due date	–
LHALF	Job with less than 50% of operations left	–
FHALF	Job with more than 50% of operations left	–
ETD	Expected tardiness	–
Unspecified	Rule not clearly specified by authors	LRPR, SRPR, CPT, TPT, TRPR, SIO, DS, LMKL, SM

review exposes prevailing trends and areas of underexplored potential. To facilitate future work, we accompany this review with an annotated dataset, supporting researchers and practitioners in exploring and building upon the surveyed literature. Overall, this review advances the transparency, comparability, and accessibility of DRL-based scheduling research.

CRediT authorship contribution statement

Constantin Waubert de Puiseau: Writing – original draft, Visualization, Methodology, Formal analysis, Data curation. **Furkan Ercan:** Writing – original draft, Visualization, Formal analysis, Data curation, Conceptualization. **Jannik Peters:** Writing – review & editing, Visualization. **Marvin Brune:** Writing – review & editing, Investigation. **Hasan Tercan:** Writing – review & editing, Project administration. **Christopher Prinz:** Writing – review & editing, Supervision. **Tobias Meisen:** Writing – review & editing, Supervision, Project administration, Funding acquisition. **Bernd Kuhlenkötter:** Writing – review & editing, Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by the German Federal Ministry of Economic Affairs and Energy through the research project Flex4Green (grant number 20D2204C).

Appendix A

See Table A.9.

References

[1] Pinedo M. Scheduling: Theory, algorithms, and systems. 6th ed. Cham: Springer International Publishing and Imprint Springer; 2022, http://dx.doi.org/10.1007/978-3-031-05921-6.

[2] Garey MR, Johnson DS, Sethi R. The complexity of flowshop and jobshop scheduling. Math Oper Res 1976;1(2):117–29. http://dx.doi.org/10.1287/moor.1.2.117.

[3] Haupt R. A survey of priority rule-based scheduling. Oper Res Spektrum 1989;11(1):3–16. http://dx.doi.org/10.1007/bf01721162.

[4] Park J, Chun J, Kim SH, Kim Y, Park J. Learning to schedule job-shop problems: Representation and policy learning using graph neural network and reinforcement learning. Int J Prod Res 2021;59(11):3360–77. http://dx.doi.org/10.1080/00207543.2020.1870013.

[5] Corsini A, Porrello A, Calderara S, Dell'Amico M. Self-labeling the job shop scheduling problem. 2024, http://dx.doi.org/10.48550/arXiv.2401.11849.

[6] Su C, Zhang C, Xia D, Han B, Wang C, Chen G, et al. Evolution strategies-based optimized graph reinforcement learning for solving dynamic job shop scheduling problem. Appl Soft Comput 2023;145:110596. http://dx.doi.org/10.1016/j.asoc.2023.110596.

[7] Gurobi. Gurobi. 2024, https://www.gurobi.com.

[8] Perron L, Furnon V. OR-tools. 2024, https://developers.google.com/optimization/.

[9] Gendreau M, Potvin J-Y. Handbook of metaheuristics, vol. 272. Cham: Springer International Publishing; 2019, http://dx.doi.org/10.1007/978-3-319-91086-4.

[10] Liu C-L, Chang C-C, Tseng C-J. Actor-critic deep reinforcement learning for solving job shop scheduling problems. IEEE Access: Pr Innov Open Solutions 2020;8:71752–62. http://dx.doi.org/10.1109/access.2020.2987820.

[11] Sabri A, Allaoui H, Souissi O. Reinforcement learning for the just-in-time job-shop scheduling problem. In: 2023 9th international conference on control, decision and information technologies. CoDIT, Rome, Italy: IEEE; 2023, p. 1367–72. http://dx.doi.org/10.1109/codit58514.2023.10284405.

[12] Song L, Li Y, Xu J. Dynamic job-shop scheduling based on transformer and deep reinforcement learning. Processes 2023;11(12):3434. http://dx.doi.org/10.3390/pr11123434.

[13] Wang S, Li J, Jiao Q, Ma F. Design patterns of deep reinforcement learning models for job shop scheduling problems. J Intell Manuf 2024. http://dx.doi.org/10.1007/s10845-024-02454-8.

[14] Khadivi M, Charter T, Yaghoubi M, Jalayer M, Ahang M, Shojaeinasab A, et al. Deep reinforcement learning for machine scheduling: methodology, the state-of-the-art, and future directions. Comput Ind Eng 2025;200:110856. http://dx.doi.org/10.1016/j.cie.2025.110856.

[15] Smit IG, Zhou J, Reijnen R, Wu Y, Chen J, Zhang C, et al. Graph neural networks for job shop scheduling problems: A survey. Comput Oper Res 2025;176:106914. http://dx.doi.org/10.1016/j.cor.2024.106914.

[16] Silver D, Hubert T, Schrittwieser J, Antonoglou I, Lai M, Guez A, et al. Mastering chess and shogi by self-play with a general reinforcement learning algorithm. 2017, http://dx.doi.org/10.48550/arXiv.1712.01815.

[17] Vinyals O, Babuschkin I, Czarnecki WM, Mathieu M, Dudzik A, Chung J, et al. Grandmaster level in StarCraft II using multi-agent reinforcement learning. Nature 2019;575(7782):350–4. http://dx.doi.org/10.1038/s41586-019-1724-z.

[18] Waubert de Puiseau C, Meyes R, Meisen T. On reliability of reinforcement learning based production scheduling systems: A comparative survey. J Intell Manuf 2022;33(4):911–27. http://dx.doi.org/10.1007/s10845-022-01915-2.

[19] Rinciog A, Meyer A. Towards standardising reinforcement learning approaches for production scheduling problems. Procedia CIRP 2022;107:1112–9. http://dx.doi.org/10.1016/j.procir.2022.05.117.

[20] Zhang C, Wu Y, Ma Y, Song W, Le Z, Cao Z, et al. A review on learning to solve combinatorial optimisation problems in manufacturing. IET Collab Intell Manuf 2023;5(1). http://dx.doi.org/10.1049/cim2.12072.

[21] Cunha B, Madureira AM, Fonseca B, Coelho D. Deep reinforcement learning as a job shop scheduling solver: A literature review. In: Hybrid intelligent systems - 18th international conference on hybrid intelligent systems, HIS 2018, Porto, Portugal, December 13-15, 2018. HIS 2018, vol. 923, 2018, p. 350–9. http://dx.doi.org/10.1007/978-3-030-14347-3_34.

[22] Shyalika C, Silva T, Karunananda A. Reinforcement learning in dynamic task scheduling: A review. SN Comput Sci 2020;1(6):306. http://dx.doi.org/10.1007/s42979-020-00326-5.

[23] Wang L, Pan Z, Wang J. A review of reinforcement learning based intelligent optimization for manufacturing scheduling. Complex Syst Model Simul 2021;1(4):257–70. http://dx.doi.org/10.23919/csms.2021.0027.

[24] Panzer M, Bender B. Deep reinforcement learning in production systems: A systematic literature review. Int J Prod Res 2022;60(13):4316–41. http://dx.doi.org/10.1080/00207543.2021.1973138.

[25] Panzer M, Bender B, Gronau N. Deep reinforcement learning in production planning and control: A systematic literature review. Hannover: publish-Ing; 2021, http://dx.doi.org/10.15488/11238.

[26] Kayhan BM, Yildiz G. Reinforcement learning applications to machine scheduling problems: A comprehensive literature review. J Intell Manuf 2023;34(3):905–29. http://dx.doi.org/10.1007/s10845-021-01847-3.

[27] Esteso A, Peidro D, Mula J, Díaz-Madroñero M. Reinforcement learning applied to production planning and control. Int J Prod Res 2023;61(16):5772–89. http://dx.doi.org/10.1080/00207543.2022.2104180.

[28] Serrano-Ruiz JC, Mula J, Poler R. Deep reinforcement learning for the job shop scheduling problem: Reference axes for modelling, implementation and validation. Lect Notes Data Eng Commun Technol 2023;160:93–100. http://dx.doi.org/10.1007/978-3-031-27915-7_18.

[29] Modrak V, Sudhakarapandian R, Balamurugan A, Soltysova Z. A review on reinforcement learning in production scheduling: An inferential perspective. Algorithms 2024;17(8):343. http://dx.doi.org/10.3390/a17080343.

[30] Dogru O, Xie J, Prakash O, Chiplunkar R, Soesanto J, Chen H, et al. Reinforcement learning in process industries: Review and perspective. IEEE/CAA J Autom Sin 2024;11(2):283–300. http://dx.doi.org/10.1109/jas.2024.124227.

[31] Zhang X, Zhu G-Y. A literature review of reinforcement learning methods applied to job-shop scheduling problems. Comput Oper Res 2025;175:106929. http://dx.doi.org/10.1016/j.cor.2024.106929.

[32] Graham RL, Lawler EL, Lenstra JK, Kan A. Optimization and approximation in deterministic sequencing and scheduling: A survey. Ann Discret Math 1979;5:287–326. http://dx.doi.org/10.1016/S0167-5060(08)70356-X.

[33] Rand GK. Machine scheduling problems: Classification, complexity and computations. European J Oper Res 1977;1(3):206. http://dx.doi.org/10.1016/0377-2217(77)90029-7.

[34] Dauzère-Pérès S, Ding J, Shen L, Tamssaouet K. The flexible job shop scheduling problem: A review. European J Oper Res 2024;314(2):409–32. http://dx.doi.org/10.1016/j.ejor.2023.05.017.

[35] Ouelhadj D, Petrovic S. A survey of dynamic scheduling in manufacturing systems. J Sched 2009;12(4):417–31. http://dx.doi.org/10.1007/s10951-008-0090-8.

[36] Dhiflaoui M, Nouri HE, Driss OB. Dual-resource constraints in classical and flexible job shop problems: A state-of-the-art review. Procedia Comput Sci 2018;126:1507–15. http://dx.doi.org/10.1016/j.procs.2018.08.123.

[37] Sutton RS, Barto A. Reinforcement learning: An introduction. 2nd ed. Adaptive computation and machine learning, Cambridge, Massachusetts London, England: The MIT Press; 2020.

[38] Bellman R. Dynamic programming. Princeton, NJ: Princeton Univ. Pr; 1957.

[39] Dulac-Arnold G, Mankowitz D, Hester T. Challenges of real-world reinforcement learning. 2019, http://dx.doi.org/10.48550/arXiv.1904.12901.

[40] Mnih V, Kavukcuoglu K, Silver D, Graves A, Antonoglou I, Wierstra D, et al. Playing atari with deep reinforcement learning. 2013, http://dx.doi.org/10.48550/arXiv.1312.5602.

[41] Achiam J. Spinning up documentation. 2018, https://spinningup.openai.com/en/latest/index.html.

[42] Silver D, Hubert T, Schrittwieser J, Antonoglou I, Lai M, Guez A, et al. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. Science 2018;362(6419):1140–4. http://dx.doi.org/10.1126/science.aar6404.

[43] Van Hasselt H, Guez A, Silver D. Deep reinforcement learning with double Q-learning. Proc AAAI Conf Artif Intell 2015;30(1). http://dx.doi.org/10.1609/aaai.v30i1.10295.

[44] Wang Z, Schaul T, Hessel M, van Hasselt H, Lanctot M, de Freitas N. Dueling network architectures for deep reinforcement learning. 2016, http://dx.doi.org/10.48550/arXiv.1511.06581.

[45] Schaul T, Quan J, Antonoglou I, Silver D. Prioritized experience replay. 2016, http://dx.doi.org/10.48550/arXiv.1511.05952.

[46] Dabney W, Ostrovski G, Silver D, Munos R. Implicit quantile networks for distributional reinforcement learning. 2018, http://dx.doi.org/10.48550/arXiv.1806.06923.

[47] Williams RJ. Simple statistical gradient-following algorithms for connectionist reinforcement learning. Mach Learn 1992;8(3–4):229–56. http://dx.doi.org/10.1007/bf00992696.

[48] Sutton RS, McAllester DA, 0001 SS, Mansour Y. Policy gradient methods for reinforcement learning with function approximation. In: Advances in neural information processing systems 12, [NIPS conference, Denver, Colorado, USA, November 29–December 4, 1999]. NIPS 1999, vol .12, 1999, p. 1057–63.

[49] Schulman J, Wolski F, Dhariwal P, Radford A, Klimov O. Proximal policy optimization algorithms. 2017, http://dx.doi.org/10.48550/arXiv.1707.06347.

[50] Mnih V, Badia AP, Mirza M, Graves A, Lillicrap TP, Harley T, et al. Asynchronous methods for deep reinforcement learning. 2016, http://dx.doi.org/10.48550/arXiv.1602.01783.

[51] Silver D, Lever G, Heess N, Degris T, Wierstra D, Riedmiller MA. Deterministic policy gradient algorithms. In: Proceedings of the 31th international conference on machine learning, ICML 2014, Beijing, China, 21-26 June 2014. ICML 2014, vol. 32, Bejing, China: PMLR; 2014, p. 387–95.

[52] Haarnoja T, Zhou A, Abbeel P, Levine S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. 2018, http://dx.doi.org/10.48550/arXiv.1801.01290.

[53] Lin L-J. Self-improving reactive agents based on reinforcement learning, planning and teaching. Mach Learn 1992;8(3–4):293–321. http://dx.doi.org/10.1007/bf00992699.

[54] Gupta D, Chandak Y, Jordan SM, Thomas PS, Da Silva BC. Behavior alignment via reward function optimization. 2023, http://dx.doi.org/10.48550/arXiv.2310.19007.

[55] Mnih V, Kavukcuoglu K, Silver D, Rusu AA, Veness J, Bellemare MG, et al. Human-level control through deep reinforcement learning. Nature 2015;518(7540):529–33. http://dx.doi.org/10.1038/nature14236.

[56] Schulman J, Levine S, Moritz P, Jordan MI, Abbeel P. Trust region policy optimization. 2017, http://dx.doi.org/10.48550/arXiv.1502.05477.

[57] Waubert de Puiseau C, Tercan H, Meisen T. Curriculum learning in job shop scheduling using reinforcement learning. Proc Conf Prod Syst Logist 2023. http://dx.doi.org/10.15488/13422.

[58] Page MJ, Moher D, Bossuyt PM, Boutron I, Hoffmann TC, Mulrow CD, et al. PRISMA 2020 explanation and elaboration: Updated guidance and exemplars for reporting systematic reviews. BMJ (Clinical Res Ed) 2021;372:n160. http://dx.doi.org/10.1136/bmj.n160.

[59] Wang L, Hu X, Wang Y, Xu S, Ma S, Yang K, et al. Dynamic job-shop scheduling in smart manufacturing using deep reinforcement learning. Comput Netw 2021;190:107969. http://dx.doi.org/10.1016/j.comnet.2021.107969.

[60] Zhou J, Zheng L, Fan W. Multirobot collaborative task dynamic scheduling based on multiagent reinforcement learning with heuristic graph convolution considering robot service performance. J Manuf Syst 2024;72:122–41. http://dx.doi.org/10.1016/j.jmsy.2023.11.010.

[61] Peters J, Waubert de Puiseau C, Tercan H, Gopikrishnan A, Lucas de Carvalho GA, Bitter C, et al. Emergent language: A survey and taxonomy. Auton Agents Multi-Agent Syst 2025;39(1):18. http://dx.doi.org/10.1007/s10458-025-09691-y.

[62] Magalhaes R, Martins M, Vieira S, Santos F, Sousa J. Encoder-decoder neural network architecture for solving job shop scheduling problems using reinforcement learning. In: 2021 IEEE symposium series on computational intelligence. SSCI, Orlando, FL, USA: IEEE; 2021, p. 1–8. http://dx.doi.org/10.1109/ssci50451.2021.9659849.

[63] Popper J, Yfantis V, Ruskowski M. Simultaneous production and AGV scheduling using multi-agent deep reinforcement learning. Procedia CIRP 2021;104:1523–8. http://dx.doi.org/10.1016/j.procir.2021.11.257.

[64] Huang D, Zhao H, Zhang L, Chen K. Learning to dispatch for flexible job shop scheduling based on deep reinforcement learning via graph gated channel transformation. IEEE Access: Pr Innov Open Solutions 2024;12:50935–48. http://dx.doi.org/10.1109/access.2024.3384923.

[65] Zeng Y, Liao Z, Dai Y, Wang R, Li X, Yuan B. Hybrid intelligence for dynamic job-shop scheduling with deep reinforcement learning and attention mechanism. 2022, http://dx.doi.org/10.48550/arXiv.2201.00548.

[66] Luo S, Zhang L, Fan Y. Real-time scheduling for dynamic partial-no-wait multiobjective flexible job shop by deep reinforcement learning. IEEE Trans Autom Sci Eng 2022;19(4):3020–38. http://dx.doi.org/10.1109/tase.2021.3104716.

[67] Hurink J, Jurisch B, Thole M. Tabu search for the job-shop scheduling problem with multi-purpose machines. Oper Res Spektrum 1994;15(4):205–15. http://dx.doi.org/10.1007/bf01719451.

[68] Taillard E. Benchmarks for basic scheduling problems. European J Oper Res 1993;64(2):278–85. http://dx.doi.org/10.1016/0377-2217(93)90182-m.

[69] Demirkol E, Mehta S, Uzsoy R. Benchmarks for shop scheduling problems. European J Oper Res 1998;109(1):137–41. http://dx.doi.org/10.1016/s0377-2217(97)00019-2.

[70] Adams J, Balas E, Zawack D. The shifting bottleneck procedure for job shop scheduling. Manag Sci 1988;34(3):391–401. http://dx.doi.org/10.1287/mnsc.34.3.391.

[71] Fischer H, Thompson GL. Probabilistic learning combinations of local job-shop scheduling rules. Prentice Hall 1963;225–51.

[72] Lawrence S. Resource constrained project scheduling. An experimental investigation of heuristic scheduling techniques (supplement). Grad Sch OfIndustrial Adm Carnegie-Mellon Univ 1984.

[73] Applegate D, Cook W. A computational study of the job-shop scheduling problem. ORSA J Comput 1991;3(2):149–56. http://dx.doi.org/10.1287/ijoc.3.2.149.

[74] Storer RH, Wu SD, Vaccari R. New search spaces for sequencing problems with application to job shop scheduling. Manag Sci 1992;38(10):1495–509. http://dx.doi.org/10.1287/mnsc.38.10.1495.

[75] Yamada T, Nakano R. A genetic algorithm applicable to large-scale job-shop problems. In: Parallel problem solving from nature 2, PPSN-II, Brussels, Belgium, September 28–30, 1992. PPSN 1992, 1992, p. 283–92.

[76] DaColDa Col G, Teppan EC. Industrial size job shop scheduling tackled by present day cp solvers. In: Principles and practice of constraint programming - 25th international conference, CP 2019, Stamford, CT, USA, September 30–October 4, 2019, proceedings. CP 2019, vol. 11802, 2019, p. 144–60. http://dx.doi.org/10.1007/978-3-030-30048-7_9.

[77] Brandimarte P. Routing and scheduling in a flexible job shop by tabu search. Ann Oper Res 1993;41(3):157–83. http://dx.doi.org/10.1007/bf02023073.

[78] Behnke D, Geiger MJ. Test instances for the flexible job shop scheduling problem with work centers (Ph.D. thesis), Hamburg: Lehrstuhl für Betriebswirtschaftslehre / Helmut-Schmidt-Universität; 2012.

[79] Fattahi P, Saidi Mehrabad M, Jolai F. Mathematical modeling and heuristic approaches to flexible job shop scheduling problems. J Intell Manuf 2007;18(3):331–42. http://dx.doi.org/10.1007/s10845-007-0026-8.

[80] Dauzère-Pérès S, Paulli J. An integrated approach for modeling and solving the general multiprocessor job-shop scheduling problem using tabu search. Ann Oper Res 1997;70:281–306. http://dx.doi.org/10.1023/a:1018930406487.

[81] Barnes JW, Chambers JB. Flexible job shop scheduling by tabu search (Ph.D. thesis), Austin: The University of Texas at Austin; 1996.

[82] Ham A. Transfer-robot task scheduling in flexible job shop. J Intell Manuf 2020;31(7):1783–93. http://dx.doi.org/10.1007/s10845-020-01537-6.

[83] Dunstall S, Wirth A. A comparison of branch-and-bound algorithms for a family scheduling problem with identical parallel machines. European J Oper Res 2005;167(2):283–96. http://dx.doi.org/10.1016/j.ejor.2004.03.022.

[84] Dunstall S, Wirth A. Heuristic methods for the identical parallel machine flowtime problem with set-up times. Comput Oper Res 2005;32(9):2479–91. http://dx.doi.org/10.1016/j.cor.2004.03.013.

[85] Liao C-J, Chao C-W, Chen L-C. An improved heuristic for parallel machine weighted flowtime scheduling with family set-up times. Comput Math Appl 2012;63(1):110–7. http://dx.doi.org/10.1016/j.camwa.2011.10.077.

[86] Sels V, Gheysen N, Vanhoucke M. A comparison of priority rules for the job shop scheduling problem under different flow time- and tardiness-related objective functions. Int J Prod Res 2012;50(15):4255–70. http://dx.doi.org/10.1080/00207543.2011.611539.

[87] Cai B, Wang S, Hu H. An effective hybrid genetic algorithm for job shop scheduling problem. Int J Adv Manuf Technol 2008;965–74. http://dx.doi.org/10.5281/zenodo.1333580.

[88] Cheng R, Gen M, Tsujimura Y. A tutorial survey of job-shop scheduling problems using genetic algorithms—I. Representation. Comput Ind Eng 1996;30(4):983–97. http://dx.doi.org/10.1016/0360-8352(96)00047-2.

[89] Goldberg DE, Deb K. A comparative analysis of selection schemes used in genetic algorithms. In: Proceedings of the first workshop on foundations of genetic algorithms. Bloomington campus, Indiana, USA, July 15–18 1990. FOGA 1990, vol. 1, 1990, p. 69–93. http://dx.doi.org/10.1016/B978-0-08-050684-5.50008-2.

[90] Blickle T, Thiele L. A comparison of selection schemes used in evolutionary algorithms. Evol Comput 1996;4(4):361–94. http://dx.doi.org/10.1162/evco.1996.4.4.361.

[91] Holland JH. Adaptation in natural and artificial systems: An introductory analysis with applications to biology, control, and artificial intelligence. Complex adaptive systems, Cambridge, Mass.: MIT Press; 1992.

[92] Zhang G, Gao L, Shi Y. An effective genetic algorithm for the flexible job-shop scheduling problem. Expert Syst Appl 2011;38(4):3563–73. http://dx.doi.org/10.1016/j.eswa.2010.08.145.

[93] Deb K, Pratap A, Agarwal S, Meyarivan T. A fast and elitist multiobjective genetic algorithm: NSGA-II. IEEE Trans Evol Comput 2002;6(2):182–97. http://dx.doi.org/10.1109/4235.996017.

[94] Defersha FM, Rooyani D. An efficient two-stage genetic algorithm for a flexible job-shop scheduling problem with sequence dependent attached/detached setup, machine release date and lag-time. Comput Ind Eng 2020;147:106605. http://dx.doi.org/10.1016/j.cie.2020.106605.

[95] Nouri HE, Driss OB, Ghédira K. A holonic multiagent model based on a combined genetic algorithm-tabu search for the flexible job shop scheduling problem. In: Advances in practical applications of agents, multi-agent systems, and sustainability: The PAAMS collection - 13th international conference, PAAMS 2015, Salamanca, Spain, June 3–4, 2015, proceedings. PAAMS 2015, vol. 524, 2015, p. 43–54. http://dx.doi.org/10.1007/978-3-319-19033-4_4.

[96] Su C-T, Shiue Y-R. Intelligent scheduling controller for shop floor control systems: A hybrid genetic algorithm/decision tree learning approach. Int J Prod Res 2003;41(12):2619–41. http://dx.doi.org/10.1080/0020754031000090612.

[97] Shiue Y-R. Data-mining-based dynamic dispatching rule selection mechanism for shop floor control systems using a support vector machine approach. Int J Prod Res 2009;47(13):3669–90. http://dx.doi.org/10.1080/00207540701846236.

[98] Chen R, Yang B, Li S, Wang S. A self-learning genetic algorithm based on reinforcement learning for flexible job-shop scheduling problem. Comput Ind Eng 2020;149:106778. http://dx.doi.org/10.1016/j.cie.2020.106778.

[99] Zhou H, Feng Y, Han L. The hybrid heuristic genetic algorithm for job shop scheduling. Comput Ind Eng 2001;40(3):191–200. http://dx.doi.org/10.1016/s0360-8352(01)00017-1.

[100] Rolf B, Reggelin T, Nahhas A, Lang S, Müller M. Assigning dispatching rules using a genetic algorithm to solve a hybrid flow shop scheduling problem. Procedia Manuf 2020;42:442–9. http://dx.doi.org/10.1016/j.promfg.2020.02.051.

[101] Akarsu CH, Küç.ükdeniz T. Job shop scheduling with genetic algorithm-based hyperheuristic approach. Int Adv Res Eng J 2022;6(1):16–25. http://dx.doi.org/10.35860/iarej.1018604.

[102] Yin L, Li X, Gao L, Lu C, Zhang Z. A novel mathematical model and multi-objective method for the low-carbon flexible job shop scheduling problem. Sustain Comput: Informatics Syst 2017;13:15–30. http://dx.doi.org/10.1016/j.suscom.2016.11.002.

[103] Momenikorbekandi A, Abbod MF. A novel metaheuristic hybrid parthenogenetic algorithm for job shop scheduling problems: Applying an optimization model. IEEE Access: Pr Innov Open Solutions 2023;11:56027–45. http://dx.doi.org/10.1109/access.2023.3278372.

[104] Li L. Research on discrete intelligent workshop lot-streaming scheduling with variable sublots under engineer to order. Comput Ind Eng 2022;165:107928. http://dx.doi.org/10.1016/j.cie.2021.107928.
[105] Ali KB, Telmoudi AJ, Gattoufi S. Improved genetic algorithm approach based on new virtual crossover operators for dynamic job shop scheduling. IEEE Access: Pr Innov Open Solutions 2020;8:213318–29. http://dx.doi.org/10.1109/access.2020.3040345.
[106] Bello I, Pham H, Le V Q, Norouzi M, Bengio S. Neural combinatorial optimization with reinforcement learning. 2016, http://dx.doi.org/10.48550/arXiv.1611.09940.
[107] Hottung A, Kwon Y-D, Tierney K. Efficient active search for combinatorial optimization problems. 2021, http://dx.doi.org/10.48550/arXiv.2106.05126.
[108] Kaven L, Huke P, Göppert A, Schmitt RH. Multi agent reinforcement learning for online layout planning and scheduling in flexible assembly systems. J Intell Manuf 2024;35(8):3917–36. http://dx.doi.org/10.1007/s10845-023-02309-8.
[109] Wang M, Zhang J, Zhang P, Xiang W, Jin M, Li H. Generative deep reinforcement learning method for dynamic parallel machines scheduling with adaptive maintenance activities. J Manuf Syst 2024;77:946–61. http://dx.doi.org/10.1016/j.jmsy.2024.11.004.
[110] Wang M, Zhang P, Zhang J, Zhang G, Bi J, Jin M. Adaptive deep reinforcement learning for dynamic parallel machines scheduling with maintenance activities. In: 2023 28th international conference on automation and computing. ICAC, Birmingham, United Kingdom: IEEE; 2023, p. 1–6. http://dx.doi.org/10.1109/icac57885.2023.10275156.
[111] Rangel-Martinez D, Ricardez-Sandoval LA. A recurrent reinforcement learning strategy for optimal scheduling of partially observable job-shop and flow-shop batch chemical plants under uncertainty. Comput Chem Eng 2024;188:108748. http://dx.doi.org/10.1016/j.compchemeng.2024.108748.
[112] Altenmüller T, Stüker T, Waschneck B, Kuhnle A, Lanza G. Reinforcement learning for an intelligent and autonomous production control of complex job-shops under time constraints. Prod Eng 2020;14(3):319–28. http://dx.doi.org/10.1007/s11740-020-00967-8.
[113] Wang Z, Liao W. Smart scheduling of dynamic job shop based on discrete event simulation and deep reinforcement learning. J Intell Manuf 2024;35(6):2593–610. http://dx.doi.org/10.1007/s10845-023-02161-w.
[114] Kuhnle A, Kaiser J-P, Theiß F, Stricker N, Lanza G. Designing an adaptive production control system using reinforcement learning. J Intell Manuf 2021;32(3):855–76. http://dx.doi.org/10.1007/s10845-020-01612-y.
[115] Zhang M, Lu Y, Hu Y, Amaitik N, Xu Y. Dynamic scheduling method for job-shop manufacturing systems by deep reinforcement learning with proximal policy optimization. Sustainability 2022;14(9):5177. http://dx.doi.org/10.3390/su14095177.
[116] Popper J, Motsch W, David A, Petzsche T, Ruskowski M. Utilizing multi-agent deep reinforcement learning for flexible job shop scheduling under sustainable viewpoints. In: 2021 international conference on electrical, computer, communications and mechatronics engineering. ICECCME, Mauritius, Mauritius: IEEE; 2021, p. 1–6. http://dx.doi.org/10.1109/iceccme52200.2021.9590925.
[117] Du Y, Li J, Li C, Duan P. A reinforcement learning approach for flexible job shop scheduling problem with crane transportation and setup times. IEEE Trans Neural Networks Learn Syst 2024;35(4):5695–709. http://dx.doi.org/10.1109/tnnls.2022.3208942.
[118] Li Y, Gu W, Yuan M, Tang Y. Real-time data-driven dynamic scheduling for flexible job shop with insufficient transportation resources using hybrid deep Q network. Robot Comput-Integr Manuf 2022;74:102283. http://dx.doi.org/10.1016/j.rcim.2021.102283.
[119] Rinciog A, Mieth C, Scheikl PM, Meyer A. Sheet-Metal Production Scheduling Using AlphaGo Zero. Hannover: publish-Ing; 2020, http://dx.doi.org/10.15488/9676.
[120] Wang S, Li J, Tang H, Wang J. CEA-FJSP: Carbon emission-aware flexible job-shop scheduling based on deep reinforcement learning. Front Environ Sci 2022;10. http://dx.doi.org/10.3389/fenvs.2022.1059451.
[121] Felder M, Steiner D, Busch P, Trat M, Sun C, Bender J, et al. Energy-flexible job-shop scheduling using deep reinforcement learning. Proc Conf Prod Syst Logist 2023. http://dx.doi.org/10.15488/13454.
[122] Turgut Y, Bozdag CE. Deep Q-network model for dynamic job shop scheduling problem based on discrete event simulation. In: 2020 winter simulation conference (WSC). Orlando, FL, USA: IEEE; 2020, p. 1551–9. http://dx.doi.org/10.1109/wsc48552.2020.9383986.
[123] Qu S, Wang J, Jasperneite J. Dynamic scheduling in large-scale stochastic processing networks for demand-driven manufacturing using distributed reinforcement learning. In: 2018 IEEE 23rd international conference on emerging technologies and factory automation, ETFA. ETFA 2018, vol. 2018-September, Turin, Italy: IEEE; 2018, p. 433–40. http://dx.doi.org/10.1109/etfa.2018.8502508.
[124] Zhang Y, Zhu H, Tang D, Zhou T, Gui Y. Dynamic job shop scheduling based on deep reinforcement learning for multi-agent manufacturing systems. Robot Comput-Integr Manuf 2022;78:102412. http://dx.doi.org/10.1016/j.rcim.2022.102412.
[125] Zhou T, Zhu H, Tang D, Liu C, Cai Q, Shi W, et al. Reinforcement learning for online optimization of job-shop scheduling in a smart manufacturing factory. Adv Mech Eng 2022;14(3). http://dx.doi.org/10.1177/16878132221086120.
[126] Chang J, Yu D, Hu Y, He W, Yu H. Deep reinforcement learning for dynamic flexible job shop scheduling with random job arrival. Processes 2022;10(4):760. http://dx.doi.org/10.3390/pr10040760.
[127] Huang S, Ontañón S. A closer look at invalid action masking in policy gradient algorithms. Int FLAIRS Conf Proc 2022;35. http://dx.doi.org/10.32473/flairs.v35i.130584.
[128] Zhang C, Song W, Cao Z, Zhang J, Tan PS, Xu C. Learning to dispatch for job shop scheduling via deep reinforcement learning. Adv Neural Inf Process Syst 2020;33:1621.1632. http://dx.doi.org/10.48550/arXiv.2010.12367.
[129] Heo CY, Seo J, Kim Y, Kim Y, Kim T. Estimated tardiness-based reinforcement learning solution to repeatable job-shop scheduling problems. Processes 2025;13(1):62. http://dx.doi.org/10.3390/pr13010062.
[130] Liao Z, Li Q, Dai Y, Zhang Z. Learning to schedule job-shop problems via hierarchical reinforcement learning. In: 2022 IEEE international conference on systems, man, and cybernetics. SMC, Prague, Czech Republic: IEEE; 2022, p. 3222–7. http://dx.doi.org/10.1109/smc53654.2022.9945585.
[131] Bonetta G, Zago D, Cancelliere R, Grosso A. Job shop scheduling via deep reinforcement learning: A sequence to sequence approach. Lecture Notes in Comput Sci 2023;475–90. http://dx.doi.org/10.1007/978-3-031-44505-7_32.
[132] Zhang S, Yin B, Cheng Y. Deep reinforcement learning for scheduling in multi-hop wireless networks : Invited paper. In: 2021 IEEE 18th international conference on mobile ad hoc and smart systems. MASS, Denver, CO, USA: IEEE; 2021, p. 1–9. http://dx.doi.org/10.1109/mass52906.2021.00010.
[133] Iklassov Z, Medvedev D, Solozabal Ochoa de Retana R, Takac M. On the study of curriculum learning for inferring dispatching policies on the job shop scheduling. In: Proceedings of the thirty-second international joint conference on artificial intelligence. IJCAI-2023, Macau, SAR China: International Joint Conferences on Artificial Intelligence Organization; 2023, p. 5350–8. http://dx.doi.org/10.24963/ijcai.2023/594.
[134] Tassel P, Gebser M, Schekotihin K. An end-to-end reinforcement learning approach for job-shop scheduling problems based on constraint programming. Proc Int Conf Autom Plan Sched 2023;33(1):614–22. http://dx.doi.org/10.1609/icaps.v33i1.27243.
[135] Lee YH, Lee S. Deep reinforcement learning based scheduling within production plan in semiconductor fabrication. Expert Syst Appl 2022;191:116222. http://dx.doi.org/10.1016/j.eswa.2021.116222.
[136] Ma R, Li F. Knowledge-specific reinforcement learning for job-shop scheduling with dynamic processing states in disassembly factory. Commun Comput Inf Sci 2024;160–9. http://dx.doi.org/10.1007/978-981-97-3948-6_16.
[137] Pirnay J, Göttl Q, Burger J, Grimm DG. Policy-based self-competition for planning problems. Eleventh Int Conf Learn Represent 2023.
[138] Su C, Zhang C, Wang C, Cen W, Chen G, Xie L. Fast Pareto set approximation for multi-objective flexible job shop scheduling via parallel preference-conditioned graph reinforcement learning. Swarm Evol Comput 2024;88:101605. http://dx.doi.org/10.1016/j.swevo.2024.101605.
[139] Xia M, Liu H, Li M, Wang L. A dynamic scheduling method with conv-dueling and generalized representation based on reinforcement learning. Int J Ind Eng Comput 2023;14(4):805–20. http://dx.doi.org/10.5267/j.ijiec.2023.6.003.
[140] Li F, Lang S, Hong B, Reggelin T. A two-stage RNN-based deep reinforcement learning approach for solving the parallel machine scheduling problem with due dates and family setups. J Intell Manuf 2024;35(3):1107–40. http://dx.doi.org/10.1007/s10845-023-02094-4.
[141] Liu Z, Mao H, Sa G, Liu H, Tan J. Dynamic job-shop scheduling using graph reinforcement learning with auxiliary strategy. J Manuf Syst 2024;73:1–18. http://dx.doi.org/10.1016/j.jmsy.2024.01.002.
[142] Burggräf P, Wagner J, Saßmannshausen T, Ohrndorf D, Subramani K. Multi-agent-based deep reinforcement learning for dynamic flexible job shop scheduling. Procedia CIRP 2022;112:57–62. http://dx.doi.org/10.1016/j.procir.2022.09.024.
[143] Vivekanandan D, Wirth S, Karlbauer P, Klarmann N. A reinforcement learning approach for scheduling problems with improved generalization through order swapping. Mach Learn Knowl Extr 2023;5(2):418–30. http://dx.doi.org/10.3390/make5020025.
[144] Thomas TE, Koo J, Chaterji S, Bagchi S. Minerva: A reinforcement learning-based technique for optimal scheduling and bottleneck detection in distributed factory operations. In: 2018 10th international conference on communication systems & networks. COMSNETS, Bengaluru: IEEE; 2018, p. 129–36. http://dx.doi.org/10.1109/comsnets.2018.8328189.
[145] Wang X, Zhang L, Lin T, Zhao C, Wang K, Chen Z. Solving job scheduling problems in a resource preemption environment with multi-agent reinforcement learning. Robot Comput-Integr Manuf 2022;77:102324. http://dx.doi.org/10.1016/j.rcim.2022.102324.
[146] Samsonov V, Kemmerling M, Paegert M, Lütticke D, Sauermann F, Gützlaff A, et al. Manufacturing control in job shop environments with reinforcement learning. In: Proceedings of the 13th international conference on agents and artificial intelligence. Online Streaming, — Select a Country —: SCITEPRESS - Science and Technology Publications; 2021, p. 589–97. http://dx.doi.org/10.5220/0010202405890597.

[147] Pol S, Baer S, Turner D, Samsonov V, Meisen T. Global reward design for cooperative agents to achieve flexible production control under real-time constraints. In: Proceedings of the 23rd international conference on enterprise information systems. Online Streaming, — Select a Country —: SCITEPRESS - Science and Technology Publications; 2021, p. 515–26. http://dx.doi.org/10.5220/0010455805150526.

[148] Samsonov V, Ben Hicham K, Meisen T. Reinforcement learning in manufacturing control: Baselines, challenges and ways forward. Eng Appl Artif Intell 2022;112:104868. http://dx.doi.org/10.1016/j.engappai.2022.104868.

[149] Raffin A, Hill A, Gleave A, Kanervisto A, Ernestus M, Dormann N. Stable-Baselines3: Reliable reinforcement learning implementations. J Mach Learn Res 2021;22(268):268:1–8.

[150] Lei K, Guo P, Wang Y, Zhang J, Meng X, Qian L. Large-scale dynamic scheduling for flexible job-shop with random arrivals of new jobs by hierarchical reinforcement learning. IEEE Trans Ind Informatics 2024;20(1):1007–18. http://dx.doi.org/10.1109/tii.2023.3272661.

[151] Zhang W, Zhao F, Li Y, Du C, Feng X, Mei X. A novel collaborative agent reinforcement learning framework based on an attention mechanism and disjunctive graph embedding for flexible job shop scheduling problem. J Manuf Syst 2024;74:329–45. http://dx.doi.org/10.1016/j.jmsy.2024.03.012.

[152] Zhao T, Wang Y, Tan Y, Zhang J. Dynamic flexible job shop scheduling algorithm based on deep reinforcement learning. In: 2023 35th Chinese control and decision conference. CCDC, Yichang, China: IEEE; 2023, p. 5099–104. http://dx.doi.org/10.1109/ccdc58219.2023.10326915.

[153] Huang J-P, Gao L, Li X-Y, Zhang C-J. A novel priority dispatch rule generation method based on graph neural network and reinforcement learning for distributed job-shop scheduling. J Manuf Syst 2023;69:119–34. http://dx.doi.org/10.1016/j.jmsy.2023.06.007.

[154] Han B-A, Yang J-J. Research on adaptive job shop scheduling problems based on dueling double DQN. IEEE Access: Pr Innov Open Solutions 2020;8:186474–95. http://dx.doi.org/10.1109/access.2020.3029868.

[155] Chen R, Li W, Yang H. A deep reinforcement learning framework based on an attention mechanism and disjunctive graph embedding for the job-shop scheduling problem. IEEE Trans Ind Informatics 2023;19(2):1322–31. http://dx.doi.org/10.1109/tii.2022.3167380.

[156] Liu C-L, Huang T-H. Dynamic job-shop scheduling problems using graph neural network and deep reinforcement learning. IEEE Trans Syst Man Cybern: Syst 2023;53(11):6836–48. http://dx.doi.org/10.1109/tsmc.2023.3287655.

[157] Cunha B, Madureira A, Fonseca B, Matos J. Intelligent scheduling with reinforcement learning. Appl Sci 2021;11(8):3710. http://dx.doi.org/10.3390/app11083710.

[158] Xu S, Li Y, Li Q. A deep reinforcement learning method based on a transformer model for the flexible job shop scheduling problem. Electronics 2024;13(18):3696. http://dx.doi.org/10.3390/electronics13183696.

[159] Liu C-L, Tseng C-J, Weng P-H. Dynamic job-shop scheduling via graph attention networks and deep reinforcement learning. IEEE Trans Ind Informatics 2024;20(6):8662–72. http://dx.doi.org/10.1109/tii.2024.3371489.

[160] Ho K-H, Cheng J-Y, Wu J-H, Chiang F, Chen Y-C, Wu Y-Y, et al. Residual scheduling: A new reinforcement learning approach to solving job shop scheduling problem. IEEE Access: Pr Innov Open Solutions 2024;12:14703–18. http://dx.doi.org/10.1109/access.2024.3357969.

[161] Liao Z, Chen J, Zhang Z. Solving job-shop scheduling problem via deep reinforcement learning with attention model. In: IEA/AIE 2023. LNAI, vol. 13926, 2023, p. 201–12. http://dx.doi.org/10.1007/978-3-031-36822-6_18.

[162] Jing X, Yao X, Liu M, Zhou J. Multi-agent reinforcement learning based on graph convolutional network for flexible job shop scheduling. J Intell Manuf 2024;35(1):75–93. http://dx.doi.org/10.1007/s10845-022-02037-5.

[163] Song W, Chen X, Li Q, Cao Z. Flexible job-shop scheduling via graph neural network and deep reinforcement learning. IEEE Trans Ind Informatics 2023;19(2):1600–10. http://dx.doi.org/10.1109/tii.2022.3189725.

[164] Park J, Bakhtiyar S, Park J. ScheduleNet: Learn to solve multi-agent scheduling problems with reinforcement learning. 2021, http://dx.doi.org/10.48550/arXiv.2106.03051.

[165] Tassel P, Gebser M, Schekotihin K. A reinforcement learning environment for job-shop scheduling. 2021, http://dx.doi.org/10.48550/arXiv.2104.03760.

[166] Lin C-C, Deng D-J, Chih Y-L, Chiu H-T. Smart manufacturing scheduling with edge computing using multiclass deep Q network. IEEE Trans Ind Informatics 2019;15(7):4276–84. http://dx.doi.org/10.1109/tii.2019.2908210.

[167] Chen X, Tian Y. Learning to perform local rewriting for combinatorial optimization. In: Advances in neural information processing systems 32: annual conference on neural information processing systems 2019, neurIPS 2019, December 8-14, 2019, Vancouver, BC, Canada. neurIPS 2019, 2019, p. 6278–89.

[168] Ho K-H, Wu J-H, Chiang F, Wu Y-Y, Chen S-I, Kuo T, et al. Deep reinforcement learning based on graph neural networks for job-shop scheduling. In: 2023 international conference on consumer electronics - Taiwan. ICCE-Taiwan, PingTung, Taiwan: IEEE; 2023, p. 805–6. http://dx.doi.org/10.1109/icce-taiwan58799.2023.10226873.

[169] Lei K, Guo P, Zhao W, Wang Y, Qian L, Meng X, et al. A multi-action deep reinforcement learning framework for flexible Job-shop scheduling problem. Expert Syst Appl 2022;205:117796. http://dx.doi.org/10.1016/j.eswa.2022.117796.

[170] Wang R, Wang G, Sun J, Deng F, Chen J. Flexible job shop scheduling via dual attention network-based reinforcement learning. IEEE Trans Neural Networks Learn Syst 2024;35(3):3091–102. http://dx.doi.org/10.1109/tnnls.2023.3306421.

[171] Yuan E, Wang L, Cheng S, Song S, Fan W, Li Y. Solving flexible job shop scheduling problems via deep reinforcement learning. Expert Syst Appl 2024;245:123019. http://dx.doi.org/10.1016/j.eswa.2023.123019.

[172] Echeverria I, Murua M, Santana R. Solving the flexible job-shop scheduling problem through an enhanced deep reinforcement learning approach. 2023, http://dx.doi.org/10.48550/arXiv.2310.15706.

[173] Zeng Z, Li X, Bai C. A deep reinforcement learning approach to flexible job shop scheduling. In: 2022 IEEE international conference on systems, man, and cybernetics. SMC, Prague, Czech Republic: IEEE; 2022, p. 884–90. http://dx.doi.org/10.1109/smc53654.2022.9945107.

[174] Han BA, Yang JJ. A deep reinforcement learning based solution for flexible job shop scheduling problem. Int J Simul Model 2021;20(2):375–86. http://dx.doi.org/10.2507/ijsimm20-2-co7.

[175] Wu X, Yan X, Guan D, Wei M. A deep reinforcement learning model for dynamic job-shop scheduling problem with uncertain processing time. Eng Appl Artif Intell 2024;131:107790. http://dx.doi.org/10.1016/j.engappai.2023.107790.

[176] Luo B, Wang S, Yang B, Yi L. An improved deep reinforcement learning approach for the dynamic job shop scheduling problem with random job arrivals. J Phys: Conf Ser 2021;1848(1):012029. http://dx.doi.org/10.1088/1742-6596/1848/1/012029.

[177] Luo S. Dynamic scheduling for flexible job shop with new job insertions by deep reinforcement learning. Appl Soft Comput 2020;91:106208. http://dx.doi.org/10.1016/j.asoc.2020.106208.

[178] Zhang L, Feng Y, Xiao Q, Xu Y, Li D, Yang D, et al. Deep reinforcement learning for dynamic flexible job shop scheduling problem considering variable processing times. J Manuf Syst 2023;71:257–73. http://dx.doi.org/10.1016/j.jmsy.2023.09.009.

[179] Luo S, Zhang L, Fan Y. Dynamic multi-objective scheduling for flexible job shop by deep reinforcement learning. Comput Ind Eng 2021;159:107489. http://dx.doi.org/10.1016/j.cie.2021.107489.

[180] Waubert de Puiseau C, Peters J, Dörpelkus C, Tercan H, Meisen T. Schlably: A Python framework for deep reinforcement learning based scheduling experiments. SoftwareX 2023;22:101383. http://dx.doi.org/10.1016/j.softx.2023.101383.

[181] Pirnay J, Grimm DG. Self-improvement for neural combinatorial optimization: Sample without replacement, but improvement. 2024, http://dx.doi.org/10.48550/arXiv.2403.15180.

[182] Berto F, Hua C, Park J, Luttmann L, Ma Y, Bu F, et al. RL4CO: An extensive reinforcement learning for combinatorial optimization benchmark. 2023, http://dx.doi.org/10.48550/arXiv.2306.17100.

[183] Reijnen R, Smit IG, Zhang H, Wu Y, Bukhsh Z, Zhang Y. Job shop scheduling benchmark: Environments and instances for learning and non-learning methods. 2023, http://dx.doi.org/10.48550/arXiv.2308.12794.

[184] Li Y, Liao C, Wang L, Xiao Y, Cao Y, Guo S. A reinforcement learning-artificial bee colony algorithm for flexible job-shop scheduling problem with lot streaming. Appl Soft Comput 2023;146:110658. http://dx.doi.org/10.1016/j.asoc.2023.110658.